차례

PART 1 전체 구조와 개발 환경

1장	웹 애플리케이션의 구조	버전 1
2장	개발 환경: 터미널, 가상환경, .env, 서버 실행	버전 1

PART 2 데이터베이스

3장	관계형 데이터베이스의 기본 개념	버전 1
4장	이음 스키마 한 줄씩 읽기	버전 1
5장	SQL: 조회, 입력, 수정, 삭제, JOIN, 집계, 트랜잭션	버전 1

PART 3 서버: Python, FastAPI, pandas

6장	main.py에 나온 Python 문법	버전 1
7장	FastAPI로 API 만들기	버전 1
8장	이음의 핵심 로직 3가지	버전 1
9장	pandas로 연구 데이터 다루기	버전 2

PART 4 규칙 · AI · 예측

10장	경고 규칙	버전 2
11장	LLM 안내 Agent	버전 2
12장	머신러닝: 추적검사 이탈 위험 예측	버전 2
13장	딥러닝: 공여자 문장 분류	버전 2

PART 5 화면과 도구

14장	HTML, CSS, JavaScript	버전 1 · 2
15장	Git과 GitHub로 이력 관리·공개하기	버전 1 · 2

PART 6 정리

16장	실제로 만난 오류와 해결	버전 1 · 2
17장	꼭 알아야 할 것 체크리스트	버전 1 · 2
18장	용어 사전	버전 1 · 2
19장	확인 문제와 정답	버전 1 · 2

버전 1 · 2 는 버전 1에 있던 장에 버전 2 내용을 더한 장입니다.

이 책을 읽는 법

이 책은 생체 신장 공여자 추적관리 앱 **이음**을 만들면서 사용한 기술을 정리한 학습서입니다. 모든 예시는 이음의 실제 코드와 데이터(공여자 D-001, D-002, D-003)입니다.

개념 정의와 원리 **꼭 알아야 할 것** 면접·실무에서 묻는 내용 **주의** 실수하면 데이터나 보안에 문제가 생김 **확인 방법** 직접 실행해 보는 법

코드는 **회색 바탕 상자**에, 코드를 한 줄씩 풀어 쓴 부분은 **왼쪽 코드 · 오른쪽 해설** 표로 정리했습니다. 처음 읽을 때는 각 장의 **꼭 알아야 할 것** 상자만 먼저 읽어도 전체 흐름을 잡을 수 있습니다.

PART 3

서버: Python, FastAPI, pandas

서버는 브라우저의 요청을 받아 입력값을 검사하고, 데이터베이스에서 데이터를 꺼내거나 저장하고, 결과를 JSON으로 돌려줍니다. 이음의 서버는 `main.py` 파일 하나(약 520줄)입니다. 이 부에서는 그 파일을 읽는 데 필요한 Python 문법과 FastAPI의 구조, 그리고 이음만의 핵심 로직을 정리합니다.

6.1	import, 변수와 자료형
6.2	리스트, 딕셔너리, 튜플
6.3	함수, 조건문, 반복문, 컴프리헨션
6.4	f-string, with, 예외, 날짜 계산
7장	FastAPI: 라우트, 경로 매개변수, Pydantic, 오류 응답, DB 연결
8장	추적검사 상태 판정, 검사 결과 자동 연결, 알림 계산
9장	pandas: DataFrame, groupby, crosstab, merge, 엑셀 내보내기

9장 pandas로 연구 데이터 다루기

pandas는 표 형태의 데이터를 Python에서 다루는 라이브러리입니다. 이음은 pandas로 두 가지를 합니다. 의료진 화면의 **최근 12주 자가 기록률**을 계산하고, 연구 데이터를 **엑셀 파일(시트 5개)**로 내보냅니다. 설치: `pip install pandas openpyxl`

9.1 DataFrame과 Series

개념 · DataFrame

행과 열로 이루어진 표 객체입니다. 각 열은 하나의 **Series**(이름이 붙은 값의 목록)이고, 각 행에는 **인덱스**(행 이름표)가 붙습니다. 데이터베이스 테이블과 모양이 같아서, SQL 결과를 그대로 담기 좋습니다.



그림 9-1 DataFrame의 구조. 예시 값은 이음 데모 데이터의 공여자 요약입니다.

SQL 결과를 DataFrame으로: query_df

코드	해설
`import pandas as pd`	pandas를 `pd` 라는 짧은 이름으로 가져온다. 관례.

`def query_df(conn, sql, columns, params=()):`
`  return pd.DataFrame(`
`  conn.execute(sql, params).fetchall(),`
`  columns=columns)`
	SQL 결과(딕셔너리의 리스트)를 표로 만든다. `columns` 로 열 이름과 순서를 정해 두면, 결과가 0행이어도 열이 있는 빈 표가 만들어져 뒤의 계산이 오류 없이 진행된다.

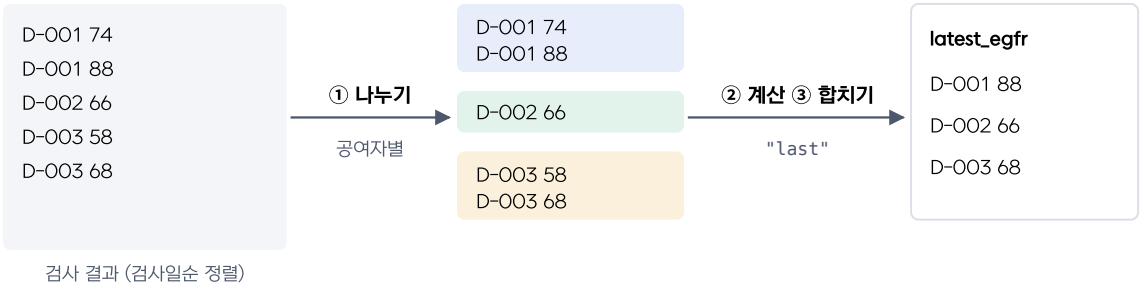
9.2 자주 쓰는 연산

코드	하는 일
`df["egfr"]`	열 하나 꺼내기 (Series)
`df[["donor_code", "egfr"]]`	여러 열 꺼내기 (DataFrame). 대괄호 두 겹
`df[df["measured_at"] >= first_week]`	조건에 맞는 행만 (SQL의 WHERE)
`df["rate"] = df["n"] / 12 * 100`	새 열 만들기. 행마다 한꺼번에 계산된다
`pd.to_numeric(df["creatinine"])`	숫자로 바꾸기. DB의 NUMERIC은 Decimal로 와서 float로 바꿔 둔다
`df.dropna(subset=["egfr"])`	egfr가 비어 있는(NaN) 행 빼기
`df["n"].fillna(0).astype(int)`	빈칸을 0으로 채운 뒤 정수로
`df["bp"].round(1)`	소수점 첫째 자리까지 반올림
`df.rename(columns={"egfr": "eGFR"})`	열 이름 바꾸기 (엑셀에 한글 머리글)
`df.empty` · `len(df)` · `df.head()`	비었는지 · 행 수 · 앞 5행 보기

개념 · NaN

pandas에서 "값 없음"을 나타내는 표시입니다(Not a Number). DB의 NULL, Python의 None이 DataFrame에 들어오면 NaN(날짜는 NaT)이 됩니다. 확인은 `pd.isna(값)` · `pd.notna(값)` 로 합니다. 평균(`mean`) 같은 집계는 NaN을 빼고 계산합니다.

9.3 groupby: 공여자별로 묶어 계산하기



검사 결과 (검사일순 정렬)

그림 9-2 `groupby`는 같은 값을 가진 행끼리 나누고, 묶음마다 계산한 뒤, 결과를 한 표로 합칩니다. SQL의 GROUP BY와 같은 생각입니다.

코드 (최근 eGFR)	해설
`latest = (labs.dropna(subset=["egfr"])`	eGFR이 없는 결과는 빼고
`.groupby("donor_code")`	공여자별로 나눠서
`.agg(latest_egfr=("egfr", "last"), latest_egfr_date=("tested_at", "last")))`	`새 열 이름=("원래 열", "계산")`. labs는 SQL에서 검사일순으로 정렬해 왔으므로 `"last"`(마지막 값) = 가장 최근 값.

계산 이름으로는 `"count"`(개수), `"mean"`(평균), `"max"`, `"min"`, `"sum"`, `"first"`, `"last"`, `"nunique"`(서로 다른 값의 개수)를 씁니다. 추적검사 완료·놓침 수는 `groupby("donor_code")["status"].value_counts().unstack(fill_value=0)`로 셉니다. `value_counts`는 값별 개수, `unstack`은 그 결과를 "공여자 × 상태" 표로 펼칩니다.

9.4 crosstab: 주간 기록 표 만들기

의료진 목록의 12주 막대와 기록률은 `weekly_records` 함수 하나에서 나옵니다. 핵심은 측정일을 **그 주의 월요일**로 바꾼 뒤, "공여자 × 주" 표로 개수를 세는 것입니다.

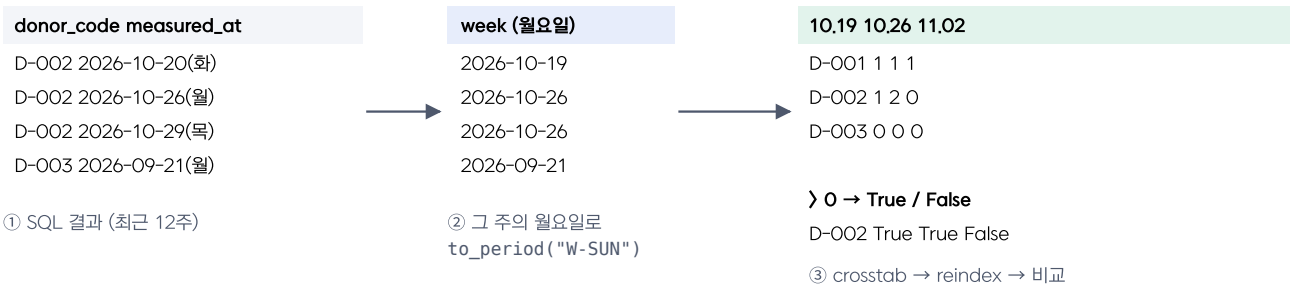


그림 9-3 같은 주에 두 번 기록해도(10.26 주 2건) 한 주로 셉니다. 기록이 없는 주는 0 → False가 됩니다.

코드 (weekly_records 핵심)	해설
`first_week = week_start(today) - timedelta(weeks=weeks - 1)`	이번 주 월요일에서 11주 전 = 12주 중 첫 주의 월요일.
`week_index = pd.date_range(first_week, periods=weeks, freq="7D")`	첫 월요일부터 7일 간격으로 12개의 날짜. 표의 열이 된다.
`if home.empty:` `  return pd.DataFrame(False, index=codes, columns=week_index)`	기록이 하나도 없으면 전부 False인 표를 바로 돌려준다(빈 표로 crosstab을 하면 열이 생기지 않음).
`home["week"] = pd.to_datetime(home["measured_at"])` `  .dt.to_period("W-SUN").dt.start_time`	`to_datetime` : 날짜 자료형으로. `"W-SUN"` : 일요일에 끝나는 주(월~일). `start_time` : 그 주의 시작 = 월요일. `.dt` 는 날짜 열에 날짜 기능을 쓰는 통로.
`counts = pd.crosstab(home["donor_code"], home["week"])`	교차표. 행 = 공여자, 열 = 주, 값 = 그 조합의 기록 건수.
`return counts.reindex(index=codes, columns=week_index,` `  fill_value=0) > 0`	`reindex` : 행과 열을 정해 둔 목록에 맞춘다. 기록이 없던 공여자·주도 0으로 채워져 항상 "전체 공여자 × 12주"가 된다. `> 0` : 칸마다 비교해 True/False 표로.

기록률은 True 칸의 비율입니다. `weekly.loc["D-002"].mean()` 은 True를 1, False를 0으로 평균 내므로 11/12 = 0.92 → 92%입니다. 의료진 화면(`/staff/overview`)과 엑셀이 같은 함수를 쓰기 때문에 두 숫자가 항상 같습니다.

9.5 merge: 표 합치기

공여자 요약 시트는 따로 계산한 표 네 개(기본 정보, 최근 eGFR, 기록률, 추적검사 수)를 `donor_code`로 이어 붙여 만듭니다. SQL의 JOIN과 같은 일입니다.

코드	해설

`summary = (donors.drop(columns="donor_id")`
`.merge(latest, on="donor_code", how="left")`
`.merge(adherence, on="donor_code", how="left")`
`.merge(visit_counts, on="donor_code", how="left"))`
	`on` : 기준 열. `how="left"` : 왼쪽(공여자 목록)의 행은 모두 남긴다(SQL의 LEFT JOIN). 검사 결과가 없는 공여자도 빠지지 않고, 그 칸은 NaN이 된다. 줄 끝마다 점(`.`)으로 이어 쓰는 것을 메서드 체이닝이라고 하며, 전체를 괄호로 감싸 줄을 나눈다.

`summary["egfr_percent"] = (summary["latest_egfr"]` `/ summary["predonation_egfr"] * 100).round()`	열끼리 계산하면 같은 행끼리 계산된다. 반복문이 필요 없다.

how	남는 행	SQL
`"inner"` (기본값)	양쪽에 모두 있는 것만	`JOIN`
`"left"`	왼쪽은 전부, 짝이 없으면 NaN	`LEFT JOIN`
`"outer"`	양쪽 전부	`FULL OUTER JOIN`

9.6 엑셀로 내보내기

코드 (export_research)	해설
`@app.get("/export/research.xlsx")`	주소가 .xlsx로 끝나 브라우저가 파일로 받는다. 의료진 화면의 "연구 데이터 엑셀 ↓" 버튼이 이 주소로 연결된다.
`buffer = io.BytesIO()`	디스크에 파일을 만들지 않고 **메모리**에 파일 내용을 쓰는 공간. 서버 폴더에 환자 데이터 파일이 쌓이지 않는다.

`with pd.ExcelWriter(buffer, engine="openpyxl") as writer:`
`  for sheet, df in tables.items():`
`  df.to_excel(writer, sheet_name=sheet, index=False)`
	엑셀 파일 하나에 표마다 시트 하나씩 쓴다. `index=False` : 0, 1, 2 행 번호 열을 빼기. 엑셀 쓰기는 openpyxl 패키지가 맡는다 (없으면 500 오류).

`ws.column_dimensions[…].width = min(width, 30)`	열 너비를 글자 수에 맞춰 조정(최대 30).
`buffer.seek(0)`	다 쓴 뒤 읽기 위치를 처음으로 되돌린다. 빠뜨리면 빈 파일이 전송된다.

`return StreamingResponse(buffer,`
`  media_type="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",`
`  headers={"Content-Disposition": f'attachment; filename="{filename}"'})`
	JSON 대신 파일 내용을 그대로 응답한다. `media_type` 은 "엑셀 파일"이라는 표시, `attachment` 는 "화면에 열지 말고 내려받기", `filename` 은 저장될 이름 (`ieum_research_20261104.xlsx`).

시트	내용
공여자 요약	기증 후 일수, 최근 eGFR과 기증 전 대비 %, 최근 12주 중 기록한 주, 기록률 %, 평균 수축기 혈압, 완료·놓친 추적검사 수
검사 결과	검사일, 크레아티닌, eGFR, 소변 단백, 연결된 추적검사, 입력자
자가 기록	측정일, 수축기·이완기 혈압, 체중
주간 기록	공여자 × 최근 12주, 기록한 주에 O
추적검사	예정일, 완료일, 상태(완료 · 놓침 · 검사 기간 · 예정)

꼭 알아야 할 것 · 설계 이유 (면접 질문 대비)

- **DataFrame** = 표, **Series** = 열 하나, **NaN** = 값 없음. SQL 결과를 표로 받아 Python에서 계산한다.
- **groupby** = 공여자별로 나눠 계산, **merge** = 기준 열로 표 합치기(how="left" = LEFT JOIN), **crosstab** = 두 열의 조합별 개수 표.
- **reindex로 빈칸 채우기**: 기록이 없는 공여자·주가 빠지면 기록률이 실제보다 높게 계산된다. 분모(12주)를 고정해야 정확하다.
- **계산 함수 하나를 화면과 엑셀이 함께 쓴다**: 같은 지표가 두 곳에서 다르게 나오는 일을 막는다.
- **메모리(BytesIO)로 엑셀 생성**: 서버에 환자 데이터 파일을 남기지 않는다. 내려받은 엑셀은 `.gitignore` 의 `*.xlsx` 로 Git에서 제외했다.
- 주간 기록 표는 나중에 ML(추적검사 이탈 위험 예측)의 입력 데이터로 그대로 쓸 수 있다.

PART 4

규칙 · AI · 예측

이음이 단순한 기록장을 넘어 "먼저 알려 주는" 도구가 되는 부분입니다. 정해 둔 기준으로 위험 신호를 찾는 **경고 규칙**, 공여자의 질문에 답하는 **LLM 안내 Agent**, 데이터에서 패턴을 배워 추적검사 이탈을 예측하는 **머신러닝**, 공여자 문장을 응급·문의·일반으로 나누는 **딥러닝**을 차례로 정리합니다.

10장	경고 규칙: 혈압 · eGFR · 단백뇨 · 체중, 순수 함수
11장	LLM 안내 Agent: 프롬프트, 질문 분류, 안전장치, 자료 주입, 속도, 스트리밍, 로컬과 클라우드
12장	머신러닝: 특징, 합성 데이터, 교차검증, AUC, 로지스틱 회귀, 서버에서 예측
13장	딥러닝: 신경망, 글자 조각, PyTorch, GroupKFold, 정밀도·재현율, numpy 추론

10장 경고 규칙

경고 규칙은 "이런 값이 나오면 의료진이 확인해야 한다"는 기준을 코드로 적어 둔 것입니다. 학습이 필요 없고, 왜 경고가 났는지 바로 설명할 수 있습니다. 이음의 기준값은 **데모용**이며 실제 운영 전에는 이식센터 의료진의 검토와 승인이 필요합니다.

10.1 규칙과 기준값

경고	조건	단계
혈압이 매우 높게 측정됨	최근 12주 중 한 번이라도 수축기 180 이상 또는 이완기 110 이상	urgent 즉시 확인
최근 2회 혈압 평균이 높음	마지막 2회 평균이 135/85 이상 (가정혈압 고혈압 기준)	warn 주의
eGFR 45 미만	가장 최근 검사의 eGFR 〈 45	urgent
eGFR이 직전보다 크게 감소	직전 검사 대비 20% 이상 감소	warn
소변 단백 양성	가장 최근 검사 1+ → 주의, 2+ · 3+ → 즉시 확인	warn · urgent
체중이 늘고 있음	12주 기간의 첫 기록 대비 5% 이상 증가	info 참고

```
# main.py 위쪽: 숫자는 이곳 한 곳에서만 바꾼다
BP_HOME_SYS, BP_HOME_DIA = 135, 85        # 가정혈압 고혈압 기준 (최근 2회 평균)
BP_CRISIS_SYS, BP_CRISIS_DIA = 180, 110   # 한 번이라도 이 이상이면 즉시 확인
EGFR_LOW = 45
EGFR_DROP_PERCENT = 20
PROTEIN_WARN = {"1+"}
PROTEIN_URGENT = {"2+", "3+"}
WEIGHT_GAIN_PERCENT = 5
```

개념 · 상수

프로그램이 실행되는 동안 바뀌지 않는 값입니다. 관례적으로 대문자 이름을 씁니다. 기준값을 코드 곳곳에 숫자로 적지 않고 상수로 모아 두면, 의료진이 기준을 바꾸자고 할 때 한 곳만 고치면 됩니다.

10.2 donor_alerts: 순수 함수로 만든 규칙

코드 (혈압 평균 규칙)	해설
`def donor_alerts(home: list[dict], labs: list[dict]) -> list[dict]:`	자가 기록과 검사 결과를 받아 경고 목록을 돌려준다. 데이터베이스를 직접 보지 않는다(순수 함수, 8.1절).
`    bp = [r for r in home if r["systolic_bp"] is not None and r["diastolic_bp"] is not None][-2:]`	혈압이 비어 있지 않은 기록만 골라 마지막 2개. `[-2:]` 는 "뒤에서 2개".
`    if len(bp) == 2: avg_sys = sum(r["systolic_bp"] for r in bp) / 2 avg_dia = sum(r["diastolic_bp"] for r in bp) / 2`	기록이 2개 있을 때만 평균을 낸다. 한 번 높게 나온 값만으로 경고하지 않으려는 것.
`        if avg_sys >= BP_HOME_SYS or avg_dia >= BP_HOME_DIA: alerts.append({"id": f"bp-high-{...}", "level": "warn", "kind": "bp", "title": "최근 2회 혈압 평균이 높음", "detail": f"평균 {avg_sys:.0f}/{avg_dia:.0f} mmHg"})`	수축기 또는 이완기 중 하나라도 기준 이상이면 경고. `:.0f` 는 소수점 없이 표시. `kind` 로 혈압·검사·체중을 구분한다.
`    order = {"urgent": 0, "warn": 1, "info": 2} return sorted(alerts, key=lambda a: order[a["level"]])`	즉시 확인 → 주의 → 참고 순으로 정렬해 돌려준다.

검사 결과 규칙은 **가장 최근 결과만** 판단합니다. 예전 결과는 이미 의료진이 확인했다고 보기 때문입니다. 기록을 읽어 오는 일은 `load_alerts(conn, donor_id, today)` 가 맡고, 판단은 `donor_alerts` 가 맡습니다. 이렇게 나누면 판단 함수를 데이터베이스 없이 시험할 수 있습니다. 실제로 9가지 경우(위기 혈압, 평균 높음, 평균 정상, 체중 증가, eGFR 낮음+감소, 감소만, 단백 1+, 예전 2+·최근 음성, 기록 없음)를 넣어 확인했습니다.

10.3 누구에게 무엇을 보여 주나

화면	보여 주는 것
의료진 목록	상태 칸 아래 경고 표시(즉시 확인 빨강 · 주의 노랑 · 참고 회색), 마우스를 올리면 실제 값. 요약 칸 "경고가 있는 공여자". 같은 상태끼리는 경고가 많은 공여자가 위로.
공여자 앱	**혈압 경고만** 알림으로. 검사 결과 해석은 의료진이 직접 전달한다.
엑셀	"공여자 요약" 시트의 "경고" 열
API	`GET /donors/{code}/alerts`

꼭 알아야 할 것 · 설계 이유

- **규칙 기반 vs 모델**: 규칙은 학습 데이터가 필요 없고 이유를 바로 설명할 수 있다. 의료 현장에서 경고는 설명 가능해야 한다.
- **기준값은 상수 한 곳**: 의료진 검토 후 숫자만 바꾸면 화면·엑셀·알림이 모두 따라 바뀐다.
- **판단(순수 함수)과 조회를 분리**: 데이터베이스 없이 경우별로 시험할 수 있다.
- **공여자에게는 혈압 경고만**: 검사 수치 해석을 앱이 직접 알리면 불안을 키울 수 있다.

11장 LLM 안내 Agent

공여자가 "다음 검사 언제예요?", "진통제 먹어도 돼요?"처럼 물으면 답하는 기능입니다. 코드는 `agent.py`에 있습니다. 핵심은 "LLM에게 모든 것을 맡기지 않는 것"입니다. 응급·일반 대화·안내문·내 기록 질문은 서버가 바로 답하고, 그 밖의 자유 질문만 LLM에 보냅니다.

11.1 LLM의 기본 개념

용어	뜻
LLM	대규모 언어 모델. 앞의 글을 보고 다음에 올 글자 조각을 하나씩 예측해 문장을 만든다.
토큰	모델이 읽고 쓰는 글자 조각의 단위. 한국어는 글자 하나가 토큰 1~2개가 되는 경우가 많다. 길이 제한과 속도는 토큰 수로 정해진다.
프롬프트	모델에 넣는 글 전체. 역할별 메시지로 나뉜다: `system`(지시문) · `user`(질문) · `assistant`(모델의 이전 답).
temperature	답의 무작위성. 0에 가까울수록 같은 질문에 같은 답. 이음은 0.1.
max tokens	답 길이의 상한. 이음은 220 (추론 모델은 1000).
할루시네이션	모델이 그럴듯하지만 사실이 아닌 내용을 지어내는 것. 의료 앱에서 가장 위험하다.
Agent · 도구 사용	모델이 "이 정보를 조회해 달라"고 도구(함수)를 요청하고, 결과를 받아 다시 답을 만드는 구조.

```
# 메시지 형식 (Claude · Groq · Ollama 공통)
[{"role": "system", "content": "너는 신장 기증자 앱 '이음'의 안내 도우미다 ..."},
 {"role": "user",   "content": "다시 회사 다녀도 될까요?"}]
```

11.2 질문 분류: AI가 필요한 질문만 AI로

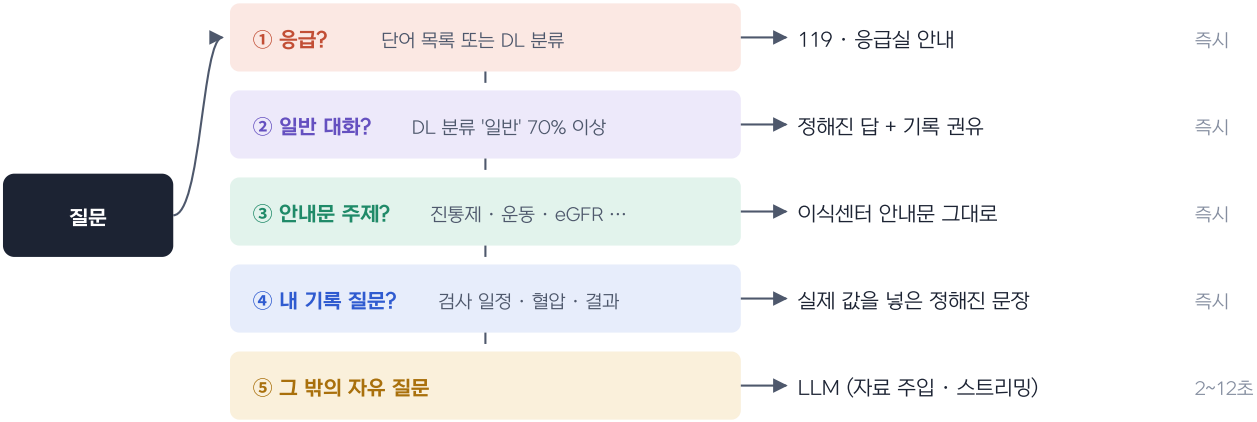


그림 11-1 위에서부터 차례로 검사해 먼저 걸리는 곳에서 답합니다. 시연에서 나올 만한 질문 대부분은 ①~④에서 처리됩니다.

```
def run_agent_stream(messages, impl):
    question = messages[-1]["content"]
    safety = safety_check(question)                  # ① 안전장치는 항상 먼저
    if safety["emergency"]: ...
    talk = small_talk(question, safety, impl)        # ② DL 분류 '일반' → 바로 답
    topic = guide_direct(question)                    # ③ 안내문 주제어
    rec = record_answer(question, impl)              # ④ 내 기록 질문
    convo, used = local_messages(messages, impl)     # ⑤ 자료를 넣어 LLM에게
```

11.3 안전장치 세 겹

안전장치	방법	막는 것
① 응급 먼저	LLM보다 먼저 `safety_check`. 단어 목록(가슴 통증, 숨이 차, 혈뇨 …) **또는** 딥러닝 분류가 응급이면 정해진 119 안내	응급 상황에서 LLM이 엉뚱하게 답하는 것
② 조회 범위 고정	조회 함수 묶음(`chat_tools`)을 서버가 공여자 코드로 고정해서 넘긴다	다른 공여자 기록이 새어 나가는 것
③ 출력 검사	답을 다 받은 뒤 "정상 범위", "괜찮아 보" 같은 판단 표현이 있으면 의료진 확인 안내로 교체	수치가 정상이라고 판단해 주는 것

```
def chat_tools(donor_code: str) -> dict:
    """Agent가 쓸 조회 함수. 공여자 코드를 여기서 고정하므로 다른 공여자의 기록은 볼 수 없다."""
    return {"get_donation_summary": lambda: donor_summary(donor_code),
            "get_home_records": lambda: list_home(donor_code)[-12:], ...}
```

시스템 지시문에는 "자료에 있는 값과 날짜만 쓸 것, 수치가 정상인지 판단하지 말 것, 진단·약 결정은 하지 말고 의료진과 상의하도록 안내할 것, 기증 후 관리와 무관한 주제는 정해진 문장으로 거절할 것"을 적었습니다.

11.4 자료 주입: 작은 모델이 값을 지어낸 문제

처음에는 Claude처럼 **도구 사용** 방식으로 만들었습니다. 그런데 노트북에서 도는 3B(30억 개 매개변수) 모델은 도구를 부르지 않고 "수축기 혈압이 120, 이완기 80, 정상 범위"라고 답했습니다. 실제 기록은 137/82였고 혈압 주의 경고가 있는 상태였습니다. 그래서 방식을 바꿨습니다.

전: 도구 사용

모델이 스스로 "혈압 기록을 조회해야겠다"고 판단해야 함 → 작은 모델은 이 판단을 자주 빠뜨리고 값을 지어냄

후: 자료 주입

서버가 다음 검사·최근 혈압·최근 검사·알림을 미리 조회해 `[자료]` 로 넣어 줌 → 모델은 문장만 만듦

```
[자료] {"기증 후 일수":208,"다음 추적검사":"기증 후 6개월 · 예정일 2026-10-10 · 지금 검사 기간 (검사 기간 마감 2026-12-09, 아직 받지 않음)",
 "최근 혈압":["2026-10-20 136/82","2026-10-27 137/82"], "최근 병원 검사":"2026-04-17 검사: eGFR 66 (기증 전 eGFR 118의 56% 수준), 소변 단백 음성",
 "지금 알림":["기증 후 6개월 검사 기간이에요", "최근 혈압이 높게 측정되고 있어요"]}
```

확인 방법 · 자료는 오해할 수 없게 쓴다

처음에는 "eGFR 66(기증 전의 56%)"라고 넣었더니 큰 모델이 "기증 전 56% 대비 상승"이라고 잘못 읽었습니다. "기증 전 eGFR 118의 56% 수준"으로 바꾸자 정확히 답했습니다. "Negative"도 "음성"으로, 검사 기간에는 마감일과 "아직 받지 않음"을 함께 적었습니다.

11.5 속도: 읽는 시간과 쓰는 시간

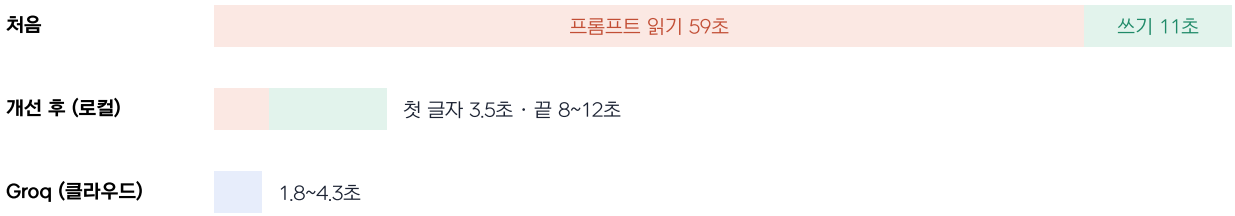


그림 11-2 시간을 재 보기 전에는 "답이 길어서 느리다"고 생각했지만, 실제 원인은 읽는 시간이었습니다.

해결책	내용
지시문·자료 줄이기	로컬 모델용 지시문을 따로 짧게 쓰고, 자료도 꼭 필요한 항목만 넣어 약 550자로 줄였다.
미리 읽혀 두기 (워밍업)	공여자 화면을 열 때 그 공여자의 지시문+자료를 모델에 미리 읽혀 둔다(답은 1글자만). Ollama는 앞부분이 같은 글을 다시 읽지 않으므로, 실제 질문에서는 질문 부분만 새로 읽는다. 모델은 30분 동안 메모리에 유지(`keep_alive`).
스트리밍	답이 다 만들어질 때까지 기다리지 않고 글자가 생기는 대로 보낸다 (11.6절).
즉시 답변 늘리기	질문 분류 ①~④를 늘려 LLM까지 가는 질문 자체를 줄였다.

11.6 스트리밍: 서버 쪽

서버는 답의 조각이 생길 때마다 **한 줄에 JSON 하나**씩 보냅니다. 이 형식을 NDJSON(줄바꿈으로 구분한 JSON)이라고 합니다.

```
{"type": "meta", "mode": "llm", "tools_used": [...]}      ← 어떤 방식으로 답하는지
{"type": "delta", "text": "현재 기록에 "}                  ← 새로 생긴 글자
{"type": "delta", "text": "따르면, "}
{"type": "replace", "mode": "guarded", "text": "..."}  ← 출력 검사에 걸리면 통째로 교체
{"type": "done"}
```

코드 (main.py)	해설
`@app.post("/donors/{donor_code}/chat/stream")` `def donor_chat_stream(donor_code: str, chat: ChatIn):`	질문(대화 기록)을 받는 POST API. `ChatIn` 은 메시지 1~20개, 각 1~1000자로 제한한 Pydantic 모델.

`    def lines():`
`        for event in agent.run_agent_stream(messages, impl):`
`            yield json.dumps(event, ensure_ascii=False) + "\n"`
	제너레이터: `return` 대신 `yield` 로 값을 하나씩 내보내는 함수. 전부 만들기 전에 앞부분부터 보낼 수 있다.

`    return StreamingResponse(lines(),` `        media_type="application/x-ndjson")`	제너레이터가 내보내는 대로 브라우저에 흘려보낸다. 9.6절의 엑셀 내보내기와 같은 응답 종류.

화면 쪽에서 이 스트림을 읽는 방법은 14.7절에서 다룹니다. LLM 서비스도 같은 방식으로 조각을 보내 주는데, 형식이 조금 다릅니다. Ollama는 `{"message": {"content": "…"}, "done": false}` 를, Groq는 `data: {"choices": [{"delta": {"content": "…"}}]}` 와 마지막 `data: [DONE]` 을 보냅니다. `llm_stream` 함수가 두 형식을 같은 글자 조각으로 바꿔 줍니다.

11.7 로컬 모델과 클라우드 모델

LLM_PROVIDER	어디서 도나	장점	단점
`ollama`	내 노트북 (qwen2.5:3b)	무료, **기록이 밖으로 나가지 않음**, 인터넷 불필요	느림(CPU), 작은 모델이라 답 품질이 낮고 범위 안의 질문도 거절하기도 함
`groq`	Groq 클라우드 (gpt-oss-120b)	무료 등급, 1.8~4.3초, 큰 모델이라 답이 자연스러움	자료가 외부로 전송됨, 인터넷 필요, 무료 한도
`anthropic`	Claude API	도구 사용 방식 그대로 가능, 품질 높음	유료, 외부 전송
`demo`	없음	설정 없이 동작	정해진 주제만 답함

```
# .env (Git에 올리지 않는다)
LLM_PROVIDER=groq
GROQ_API_KEY=gsk_...                  # Groq 키는 gsk_ 로 시작
# GROQ_MODEL=openai/gpt-oss-120b     # 생략하면 이 모델
```

Groq, Ollama, Claude는 모두 같은 메시지 형식을 쓰므로 `.env` 한 줄로 바꿀 수 있습니다. Groq 요청은 Python 표준 라이브러리 `urllib` 로 보내고, `Authorization: Bearer 키` 헤더로 인증합니다. 추론 모델(gpt-oss)은 답하기 전에 생각을 먼저 하므로, 생각은 짧게(`reasoning_effort: low`) 하고 화면에는 내보내지 않게(`include_reasoning: false`) 설정했습니다.

11.8 연결 문제 찾기: /health

```
{"llm_provider": "groq", "llm_model": "openai/gpt-oss-120b", "groq_key_set": true,
 "groq_key_check": {"length": 56, "starts_with_gsk_": true, "has_space_or_quote": false},
 "llm_last_error": null}
```

겪은 문제	/health에서 보인 것	해결
.env를 바꿨는데 계속 로컬 모델로 답함	`llm_provider: "ollama"`	`.env`의 `LLM_PROVIDER=ollama` 줄을 `groq`로. `load_dotenv(override=True)`로 .env가 항상 우선하게
키를 넣었는데 연결 실패	`HTTP 401 Invalid API Key`, `starts_with_gsk_: false`	Groq가 아닌 다른 곳의 키였음 → console.groq.com에서 다시 발급
키는 맞는데 연결 실패	`HTTP 404 model_not_found`	설정한 모델이 서비스 종료됨 → 쓸 수 있는 모델 목록을 조회해 자동으로 바꾸는 기능 추가

키 값 자체는 어디에도 표시하지 않고 **모양(길이, 앞부분, 공백 여부)만** 확인합니다. 연결 오류는 서버 터미널에도 `[agent] AI 연결 실패: …`로 남깁니다.

꼭 알아야 할 것 · 설계 이유 (면접 질문 대비)

- **LLM에 모든 것을 맡기지 않는다**: 응급·일반 대화·안내문·내 기록은 서버가 정해진 방식으로 즉시 답하고, 자유 질문만 LLM으로.
- **할루시네이션 대응**: 직접 확인한 문제(작은 모델이 값을 지어냄)를 자료 주입 + 출력 검사로 막았다.
- **안전장치 세 겹**: 응급 먼저(단어+DL), 조회 범위 고정, 출력 검사.
- **측정 후 개선**: 느린 원인이 "읽기"임을 재서 확인 → 첫 응답 59초 → 3.5초.
- **개인정보**: 기본은 기록이 밖으로 나가지 않는 로컬 모델, 시연은 클라우드. 실제 병원 적용에는 정보보호 승인 필요.

12장 머신러닝: 추적검사 이탈 위험 예측

"이 공여자가 다음 추적검사를 놓칠 확률은 얼마인가"를 예측합니다. 규칙(10장)은 사람이 기준을 정하지만, 머신러닝은 **데이터에서 기준을 배웁니다**. 실제 환자 데이터가 없어서 **합성 데이터**로 학습했으므로, 이 모델은 전체 흐름을 보여 주는 시연용입니다.

12.1 지도학습과 분류

개념 · 지도학습 (supervised learning)

입력값(특징, feature)과 정답(라벨, label)이 짝지어진 데이터로 학습해, 새 입력의 정답을 예측하는 방법입니다. 정답이 "놓침 1 / 받음 0"처럼 범주이면 **분류**, 숫자이면 **회귀**라고 합니다. 이음의 이탈 예측은 이진 분류이고, 모델은 0~1 사이 확률을 냅니다.

특징 (ml/features.py)	뜻
`record_rate_12w`	최근 12주 자가 기록률 (0~1, 9.4절의 주간 기록 표)
`weeks_since_home`	마지막 자가 기록 후 지난 주 수 (기록 없으면 26)
`prior_missed`	지금까지 놓친 추적검사 수
`mean_delay_days`	지금까지 받은 검사가 예정일보다 평균 며칠 늦었나
`years_since_donation`	기증 후 햇수
`age`	나이
`bp_alert`	혈압 경고가 있으면 1

특징 이름과 순서를 `ml/features.py` 한 파일에서 정해, 학습할 때와 서버에서 예측할 때 똑같이 씁니다. 순서가 어긋나면 계수가 엉뚱한 값과 곱해지기 때문입니다.

12.2 합성 데이터

`ml/make_synthetic.py`가 가상의 공여자 2,000명을 만듭니다. 눈에 보이지 않는 "관리 참여도"를 정해 두고, 참여도가 낮을수록 기록률이 낮고 끊긴 기간이 길고 이전에 놓친 검사가 많도록 값을 만든 뒤, 가정한 규칙과 우연(노이즈)으로 정답(다음 검사를 놓쳤는지)을 정했습니다. 놓침 비율은 32.4%입니다.

주의 · 합성 데이터의 한계

모델은 **제가 넣은 가정을 다시 찾아낼 뿐**입니다. 성능 수치는 "학습·평가·서비스 연결이 제대로 동작한다"는 확인이지, 실제 공여자에게 맞는다는 근거가 아닙니다. 실제 적용에는 실제 추적 데이터, IRB 승인, 가명처리, 외부 검증이 필요합니다.

개념 · 재현성과 seed

`np.random.default_rng(42)` 처럼 난수의 시작점(seed)을 고정하면 어느 컴퓨터에서 실행해도 같은 데이터와 같은 결과가 나옵니다. 실제로 PC와 서버에서 실행한 결과가 소수점까지 같았습니다. 연구 데이터를 다룰 때 중요한 성질입니다.

12.3 학습과 평가

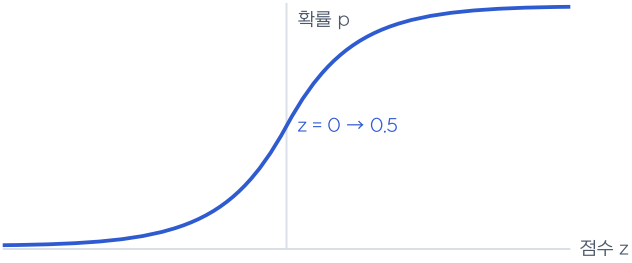
코드 (ml/train.py)	해설
`X_train, X_test, y_train, y_test = train_test_split(` `  X, y, test_size=0.25, stratify=y, random_state=42)`	75%는 학습용, 25%는 평가용으로 나눈다. `stratify=y` : 놓침 비율이 양쪽에 같게.
`"logistic": make_pipeline(StandardScaler(), LogisticRegression())`	표준화 → 로지스틱 회귀를 한 덩어리(파이프라인)로.
`"random_forest": RandomForestClassifier(n_estimators=300,` `  min_samples_leaf=10)`	비교용: 결정 나무 300개의 다수결.
`cv_auc = cross_val_score(model, X_train, y_train,` `  cv=5, scoring="roc_auc")`	5겹 교차검증: 학습용을 5조각으로 나눠 4조각으로 학습하고 1조각으로 평가하기를 5번.
`model.fit(X_train, y_train)` `p = model.predict_proba(X_test)[:, 1]`	학습(`fit`) 후, 평가용에서 "놓침(1)"일 확률을 뽑는다.

지표	뜻
AUC	놓칠 사람에게 놓치지 않을 사람보다 더 높은 위험을 줄 확률. 0.5는 찍기, 1.0은 완벽.
교차검증 ± 값	5번 평가의 흔들림(표준편차). 작을수록 안정적.
Brier 점수	예측 확률과 실제 결과(0/1)의 차이를 제곱해 평균 낸 값. 작을수록 좋음.
과적합	학습 데이터에만 맞고 새 데이터에서는 틀리는 상태. 교차검증과 평가용 결과가 비슷하면 과적합이 없다고 본다.

모델	교차검증 AUC	평가용 AUC	Brier
로지스틱 회귀	**0.812** ± 0.030	0.803	0.164
랜덤 포레스트	0.801 ± 0.032	0.797	0.164

성능이 비슷하므로 **설명하기 쉬운 로지스틱 회귀**를 골랐습니다. 의료진에게 "왜 위험이 높은가"를 보여 줄 수 있기 때문입니다.

12.4 로지스틱 회귀와 계수 해석



① 표준화: (값 − 평균) ÷ 표준편차

② 점수 z = 절편 + Σ (계수 × 표준화 값)

③ 확률 p = 1 ÷ (1 + e^{-z}) (시그모이드)

④ 20% 미만 낮음 · 40% 이상 높음

그림 12-1 로지스틱 회귀는 점수를 시그모이드로 0~1 확률로 바꿉니다.

특징	계수	해석 (합성 데이터 기준)
이전에 놓친 검사 수	+0.89	가장 큰 요인. 한 번 놓친 사람이 또 놓침
최근 12주 기록률	−0.50	기록을 꾸준히 할수록 위험이 내려감
기록이 끊긴 주 수	+0.39	오래 끊길수록 위험이 올라감
기증 후 햇수 · 나이	+0.24 · −0.24	시간이 지날수록, 젊을수록 위험이 높음
평균 지연일 · 혈압 경고	+0.20 · +0.05	영향이 작음

표준화한 뒤의 계수라서 서로 크기를 비교할 수 있습니다. +는 위험을 올리고 −는 내립니다.

12.5 서버에서 예측: risk.py

학습은 scikit-learn으로 하지만, 서버에는 결과(평균·표준편차·계수·절편)만 `ml/dropout_model.json`으로 저장해 씁니다. 덧셈·곱셈과 시그모이드만 있으면 되므로 **서버에 scikit-learn이 필요 없습니다.**

코드 (risk.py · predict)	해설
`z = [(features[f] - m) / s for f, m, s in zip(FEATURES, MODEL["mean"], MODEL["scale"])]`	① 표준화. `zip`은 여러 목록을 같은 순서끼리 묶어 꺼낸다.
`contrib = {f: c * v for f, c, v in zip(FEATURES, MODEL["coef"], z)}`	특징마다 "계수 × 표준화 값" = 그 특징이 점수에 기여한 양.
`logit = MODEL["intercept"] + sum(contrib.values())` `p = 1 / (1 + math.exp(-logit))`	② 점수, ③ 시그모이드로 확률.
`reasons = [LABELS_KO[f] for f, c in sorted(contrib.items(), key=lambda kv: -kv[1]) if c > 0.15][:2]`	기여가 큰 순으로 정렬해 위험을 올린 이유 2개를 한국어로. 의료진 화면에 마우스를 올리면 보인다.

공여자	예측 대상	이탈 위험	주요 이유
D-003	기증 후 3년	**49% · 높음**	이전에 놓친 추적검사, 자가 기록이 끊긴 기간
D-001	기증 후 2년	14% · 낮음	이전에 놓친 추적검사
D-002	기증 후 6개월	5% · 낮음	두드러진 요인 없음

꼭 알아야 할 것

- 지도학습 = 특징 + 정답으로 학습. 이탈 예측은 이진 분류이고 결과는 확률.
- 평가는 학습에 쓰지 않은 데이터로. 교차검증과 평가용 결과가 비슷하면 과적합이 없다.
- AUC는 순서를 맞히는 능력, Brier는 확률이 얼마나 정확한가.
- 성능이 비슷하면 설명 가능한 모델을 고른다. 계수 × 값으로 이유를 보여 줄 수 있다.
- 합성 데이터 결과는 파이프라인 확인일 뿐, 실제 성능이 아니다.

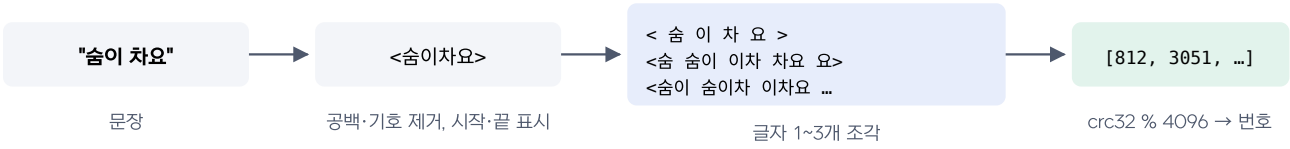
13장 딥러닝: 공여자 문장 분류

공여자가 쓴 문장을 **응급 / 문의 / 일반**으로 나누는 작은 신경망입니다. 응급 단어 목록에 없는 표현("갑자기 오른팔에 힘이 안 들어가요")을 잡아내고, 일상 대화("오늘 산책했어요")를 LLM 없이 바로 답하게 하는 데 씁니다. PyTorch로 학습하고, 서버에서는 numpy로 계산합니다.

13.1 신경망의 기본 개념

용어	뜻
층 (Linear)	입력 벡터에 가중치 행렬을 곱하고 편향을 더한다: `W·x + b`. 학습으로 W와 b가 정해진다.
활성화 함수 (ReLU)	음수는 0, 양수는 그대로: `max(0, x)`. 층 사이에 넣어 직선으로 표현할 수 없는 관계를 배우게 한다.
softmax	마지막 점수 3개를 합이 1인 확률 3개로 바꾼다.
손실 함수	예측이 정답에서 얼마나 틀렸는지를 숫자로. 분류에는 교차 엔트로피(CrossEntropyLoss).
역전파 · 옵티마이저	손실을 줄이는 방향으로 가중치를 조금씩 고친다. 이음은 AdamW (학습률 0.01).
에폭 · 배치	에폭 = 전체 데이터를 한 번 다 보는 것(40번). 배치 = 한 번에 고치는 데 쓰는 문장 묶음(32개).
드롭아웃	학습 중에만 일부 값(20%)을 무작위로 0으로 만들어, 특정 값에만 기대지 않게 한다.

13.2 문장을 숫자로: 글자 조각과 해시



띄어쓰기("숨이 차요" / "숨이차요")와 말끝("차요" / "차")이 달라도 겹치는 조각이 많아 같은 뜻으로 묶인다.
파이썬 hash()는 실행할 때마다 값이 바뀌므로, 항상 같은 값을 내는 crc32를 쓴다.

그림 13-1 `ml/triage_features.py`의 처리 과정. 학습과 서버가 같은 파일을 씁니다.

13.3 모델 구조와 PyTorch 코드

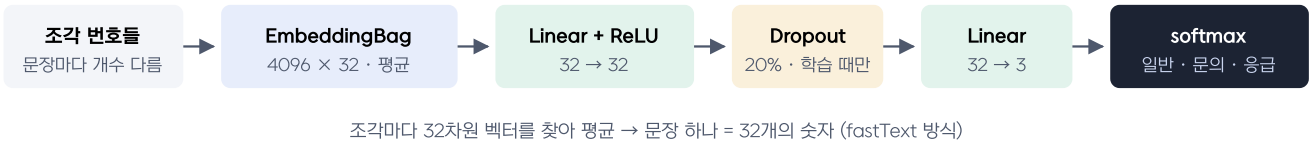


그림 13-2 이음의 문장 분류 모델. 가중치는 약 13만 개로 작습니다.

코드 (ml/train_triage.py)	해설

`class TriageNet(nn.Module):`
`  def __init__(self, buckets=4096, dim=32, hidden=32, n_classes=3):`
`  super().__init__()`
`  self.emb = nn.EmbeddingBag(buckets, dim, mode="mean")`
`  self.fc1 = nn.Linear(dim, hidden)`
`  self.drop = nn.Dropout(0.2)`
`  self.fc2 = nn.Linear(hidden, n_classes)`
	PyTorch 모델은 `nn.Module`을 상속한 클래스. `__init__`에서 층을 만든다. `super().__init__()`은 부모 클래스의 준비 작업을 먼저 실행.

`  def forward(self, ids, offsets):`
`  h = self.emb(ids, offsets)`
`  h = torch.relu(self.fc1(h))`
`  return self.fc2(self.drop(h))`
	`forward` : 입력이 층을 지나가는 순서. softmax는 손실 함수 안에서 계산하므로 여기서는 점수만 돌려준다.

`weights = len(train_idx) / (3 * counts)` `loss_fn = nn.CrossEntropyLoss(weight=weights)`	라벨 수가 다르므로(일반 150 · 문의 352 · 응급 266) 적은 라벨에 가중치를 더 준다.

`loss = loss_fn(model(ids, offsets), labels)`
`opt.zero_grad()`
`loss.backward()`
`opt.step()`
	학습 한 걸음: 예측 → 손실 계산 → 이전 기울기 지우기 → 역전파로 기울기 계산 → 가중치 수정.

`EmbeddingBag`은 문장마다 조각 개수가 다른 것을 처리하려고, 모든 문장의 조각 번호를 한 줄로 이어 붙이고(`ids`) 각 문장이 시작하는 위치(`offsets`)를 함께 받습니다.

13.4 평가: 같은 틀이 섞이지 않게

학습 문장 768개는 문장 틀 47개에서 단어와 말끝을 바꿔 만들었습니다(`ml/triage_data.py`). 같은 틀에서 나온 문장이 학습과 평가에 함께 들어가면, 모델은 처음 보는 표현이 아니라 거의 같은 문장을 맞히게 되어 점수가 부풀려집니다. 그래서 **틀 단위로 나누는 GroupKFold**(5겹)로 평가했습니다. 처음 이 방식을 적용했을 때 점수가 크게 떨어져, 틀을 30개에서 47개로 늘리고 틀마다 문장 수를 24개 이하로 맞췄습니다.

	딥러닝	로지스틱 회귀
정확도	0.798	**0.831**
응급 정밀도	0.872	0.872
응급 재현율	0.718	0.718

정답 \ 예측	일반	문의	응급
일반	115	19	16
문의	33	307	12
응급	23	**52**	191

혼동 행렬에서 가장 큰 오류는 **응급 문장 52개를 문의로 본 것**입니다. "혈압이 190/120 나왔고 머리가 아파요"(응급)와 "혈압이 142/90 나왔는데 괜찮을까요"(문의)는 글자 조각이 많이 겹치기 때문입니다. 데이터가 작고 구조가 단순해서 딥러닝이 로지스틱 회귀보다 낫지 않았습니다. 그래서 응급 판단은 **단어 목록과 모델을 함께** 쓰는 구조로 설계했습니다.

13.5 정밀도, 재현율, 기준값

지표	뜻	응급에서는
정밀도	응급이라고 한 것 중 실제 응급의 비율	낮으면 오경보가 많음
재현율	실제 응급 중 응급이라고 잡아낸 비율	낮으면 응급을 놓침 → **더 중요**

응급 확률 기준	응급 재현율	응급이 아닌데 응급으로 본 비율
0.50	0.707	0.054
0.35 (현재)	0.737	0.066
0.25	0.767	0.074

가장 높은 확률을 고르는 대신 "응급 확률이 기준 이상이면 응급"으로 정해, 기준을 낮출수록 응급을 덜 놓치는 대신 오경보가 늘어납니다. 놓치는 비용이 훨씬 크므로 0.5보다 낮은 값을 씁니다.

13.6 서버에서 계산: triage.py

코드 (numpy)	해설
`W = np.load("ml/triage_model.npz")`	PyTorch로 학습한 가중치(emb, w1, b1, w2, b2)를 numpy 배열로 저장해 둔 파일.
`h = W["emb"][ngram_ids(text)].mean(axis=0)`	EmbeddingBag(mean)과 같은 계산: 조각 번호의 행을 꺼내 평균.
`h = np.maximum(W["w1"] @ h + W["b1"], 0)`	Linear + ReLU. `@`는 행렬 곱. 드롭아웃은 학습 때만 쓰므로 생략.
`logits = W["w2"] @ h + W["b2"]` `p = np.exp(logits - logits.max()); p = p / p.sum()`	softmax. 가장 큰 값을 먼저 빼는 것은 큰 수의 exp가 넘치지 않게 하는 관례.

서버에 PyTorch(약 200MB)를 설치하지 않아도 됩니다. 처음에는 같은 구조를 numpy로 직접 구현해 학습한 임시본을 쓰다가, PyTorch로 다시 학습했을 때 정확도가 0.801 → 0.798로 거의 같아 두 계산이 일치함을 확인했습니다.

13.7 안내 Agent에서 쓰는 곳

문장	단어 목록	DL 분류	결과
갑자기 오른팔에 힘이 안 들어가요	못 잡음	응급	119 안내 (단어 목록을 보완)
소변 색이 이상하게 빨개요	못 잡음	응급	119 안내
오늘 산책했어요	-	일반	0.3초 안에 답 + 이번 주 기록 권유

꼭 알아야 할 것

- 신경망 = 층(W·x+b) + 활성화(ReLU) 반복, 마지막 softmax. 손실을 역전파로 줄이며 학습.
- 한국어 짧은 문장은 글자 1~3개 조각이 띄어쓰기·말끝 차이에 강하다.
- 같은 틀·같은 사람의 데이터가 학습과 평가에 섞이면 점수가 부풀려진다 → GroupKFold.
- 응급 분류에서는 정밀도보다 재현율이 중요하고, 기준값으로 둘을 조절한다.
- 딥러닝이 항상 낫지 않다. 같은 방법으로 비교해 결과를 그대로 보고하고, 설계(단어+모델)로 보완한다.

PART 5

화면과 도구

브라우저 화면은 세 가지 언어로 만듭니다. HTML은 구조, CSS는 모양, JavaScript는 동작입니다. 이음의 화면 세 개는 각각 HTML 파일 하나에 세 언어가 모두 들어 있습니다. 마지막으로 코드의 변경 이력을 남기는 Git과, 포트폴리오로 공개하는 GitHub를 정리합니다.

14.1	HTML: 태그와 속성, 폼, 대화상자
14.2	CSS: 선택자, 변수, 그리드, 반응형
14.3	JavaScript: 변수, 함수, DOM, 이벤트
14.4	fetch와 async/await: 서버와 통신
14.5	저장 버튼을 누르면 일어나는 일
14.6	Chart.js, localStorage, Notification
14.7	스트리밍 응답을 화면에서 읽기
15장	Git과 GitHub: 세 영역, 명령, .gitignore, 원격 저장소, 브랜치, README

14장 HTML, CSS, JavaScript

브라우저는 서버에서 받은 HTML을 읽어 화면의 뼈대를 만들고, CSS로 꾸미고, JavaScript를 실행해 데이터를 불러오고 버튼에 반응합니다.

14.1 HTML

```
<button id="openForm" class="primary">기록하기</button>
 └ 여는 태그      └ 속성(이름="값")              └ 내용    └ 닫는 태그
```

태그	의미
`<head>` · `<body>`	문서 정보(제목, CSS 연결) · 화면에 보이는 내용
`<div>` · `<span>`	의미 없는 묶음. div는 한 줄 전체(블록), span은 글자 사이(인라인)
`<h1>`~`<h6>` · `<p>`	제목 · 문단
`<table>` `<tr>` `<th>` `<td>`	표 · 행 · 제목 칸 · 칸 (의료진 목록)
`<form>` `<input>` `<label>`	입력 양식. `<input type="number" name="systolic_bp" min="50" max="260" required>`
`<dialog>`	팝업 창. JS로 `showModal()` · `close()`
`<canvas>` · `<svg>`	그림판(Chart.js 그래프) · 벡터 도형(신장 일러스트)
`<script>` · `<style>` · `<link>`	JS 코드 · CSS 코드 · 외부 CSS 파일 연결

id는 문서 안에서 하나뿐인 이름(JS가 찾을 때), **class**는 여러 요소에 붙일 수 있는 이름(CSS로 꾸밀 때)입니다. input의 `min`, `max`, `required`는 브라우저가 먼저 검사하는 규칙이지만, 브라우저 검사는 우회할 수 있으므로 서버의 Pydantic 검사가 반드시 따로 있어야 합니다.

14.2 CSS

코드	해설
`:root { --blue: #3F6FE0; }` `.badge { color: var(--blue); }`	`--이름`은 CSS 변수. 색을 한 곳에서 정하고 `var()`로 재사용한다.
`.card { padding: 16px; border-radius: 14px; }`	`.이름` = class 선택자. padding 안쪽 여백, border-radius 모서리 둥글기.
`#hello { font-size: 22px; }`	`#이름` = id 선택자.
`.step.missed { background: var(--red-soft); }`	class가 둘 다 있는 요소. JS가 상태에 따라 class를 붙이면 색이 바뀐다.
`.grid { display: grid; gap: var(--gap);` `grid-template-columns: minmax(0, 1fr); }`	그리드 배치. 기본은 한 줄(휴대폰). `minmax(0, 1fr)`은 내용이 길어도 칸이 화면 밖으로 밀려나지 않게 한다(`1fr`만 쓰면 긴 표나 그래프가 칸을 넓힌다).

`@media (min-width: 1001px) {`
`.grid { grid-template-columns:`
`minmax(0, 1.1fr) minmax(0, 1.15fr) minmax(0, .85fr); } }`
	미디어 쿼리. 화면 폭이 1001px 이상이면 세 칸으로 나눈다(비율 1.1 : 1.15 : 0.85). 화면 크기에 맞춰 배치가 바뀌는 것을 반응형이라고 한다.

`th, td { white-space: nowrap; }`	표 칸의 글자를 줄바꿈하지 않는다(의료진 목록).
`@keyframes flow { to { stroke-dashoffset: -400; } }`	애니메이션 정의. 대시보드 혈관의 흐르는 빛.

14.3 JavaScript 기본

코드	해설
`const DAY = 86400000;` `let bpChart;`	`const`는 다시 대입할 수 없는 변수, `let`은 바꿀 수 있는 변수. 하루 = 86,400,000밀리초.
`const $ = (id) => document.getElementById(id);`	화살표 함수. `(매개변수) => 결과`. id로 HTML 요소를 찾는 짧은 이름 `$`를 만든다.
`$("hello").textContent = "기증 후 506일째";`	DOM 조작. DOM은 HTML을 JS에서 다룰 수 있게 만든 객체 구조. 요소의 글자를 바꾼다.
`$("steps").innerHTML = ` ${name}</div>`;`	HTML을 통째로 넣는다. 백틱(`` ` ``) 문자열 안의 `${값}`은 Python의 f-string과 같다(템플릿 리터럴).

`$("openForm").addEventListener("click", () => {`
`  $("formDialog").showModal();`
`});`
	이벤트. "클릭되면 이 함수를 실행". 폼은 `"submit"`, 선택 상자는 `"change"`.

`s.visits.find((v) => v.status === "window")`	배열에서 조건에 맞는 첫 원소. 없으면 `undefined`. `===`는 값과 종류가 모두 같음.
`home.map((r) => mondayOf(r.measured_at))`	각 원소를 바꾼 새 배열. Python의 `[f(r) for r in home]`.
`home.filter((r) => r.systolic_bp != null)`	조건에 맞는 원소만. Python의 `[r for r in home if …]`.
`last?.egfr ?? "-"`	`?.`는 last가 없으면 오류 대신 undefined. `??`는 왼쪽이 null/undefined면 오른쪽 값.

14.4 fetch와 async/await

개념 · 비동기 (asynchronous)

서버 응답은 바로 오지 않습니다. 기다리는 동안 화면이 멈추지 않도록, JS는 요청을 보내 놓고 응답이 오면 이어서 실행합니다. `async`를 붙인 함수 안에서 `await`를 쓰면 "응답이 올 때까지 이 줄에서 기다린다"를 위에서 아래로 읽히게 쓸 수 있습니다.

코드 (donor.html · loadAll 일부)	해설
`async function loadAll() {` `  try {`	오류가 날 수 있는 부분을 `try`로 감싼다.
`    const c = encodeURIComponent(code());`	주소에 넣을 수 있게 특수문자를 변환.

`    const [sRes, hRes, lRes, nRes] = await Promise.all([`
`      fetch(`/donors/${c}/summary`), fetch(`/donors/${c}/home`),`
`      fetch(`/donors/${c}/labs`), fetch(`/donors/${c}/notifications`),`
`    ]);`
	GET 요청 4개를 **동시에** 보내고 모두 올 때까지 기다린다. 결과를 순서대로 변수 4개에 나눠 담는다(구조 분해).

`    if (!sRes.ok) { … return; }`	`res.ok`는 상태 코드가 200번대일 때 true. 404·500이면 메시지를 보여주고 중단.
`    const s = await sRes.json();`	응답 본문(JSON 글자)을 JS 객체로 바꾼다. 이것도 기다려야 한다.
`    renderHello(s); renderWeek(home); …`	받은 데이터로 화면의 각 부분을 그린다.

`  } catch {`
`    $("hello").textContent = "서버에 연결할 수 없습니다";`
`  }`
	서버가 꺼져 있으면 fetch 자체가 실패해 여기로 온다. 404·500은 응답이 온 것이므로 catch로 오지 않는다.

14.5 저장 버튼을 누르면 일어나는 일

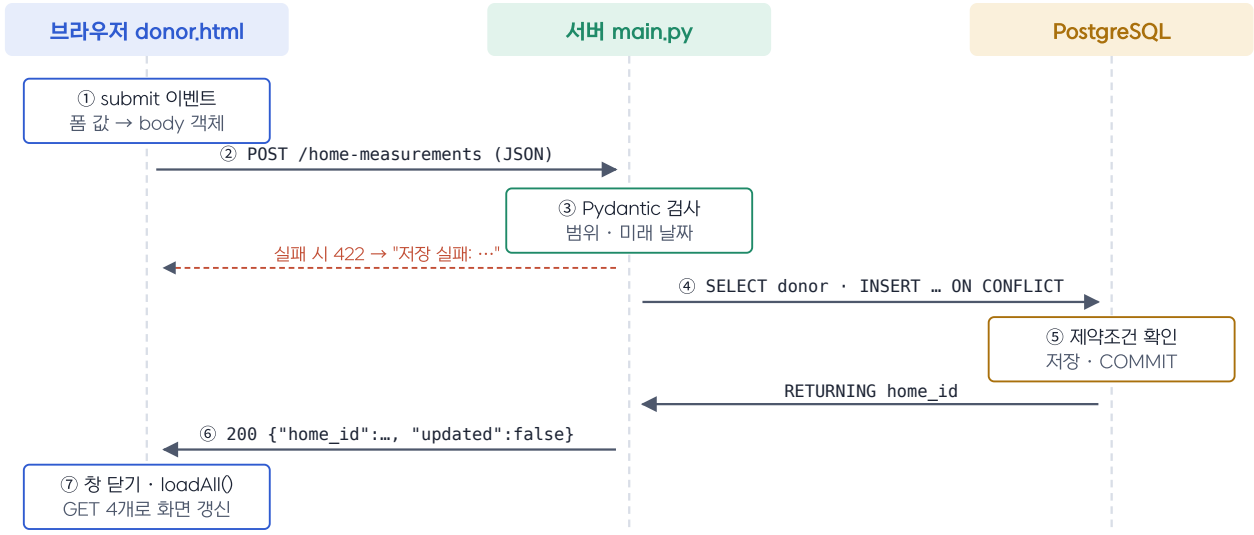


그림 14-1 기록 저장의 전체 순서. 검사는 브라우저(input 규칙), 서버(Pydantic), 데이터베이스(제약조건) 세 번 이루어집니다.

코드 (donor.html · 기록 저장)	해설
`$("form").addEventListener("submit", async (e) => {` `  e.preventDefault();`	폼 제출 시 브라우저의 기본 동작(페이지 새로고침)을 막는다.

`  const f = new FormData(e.target);`
`  const body = {`
`  donor_code: code(), measured_at: f.get("measured_at"),`
`  systolic_bp: num(f.get("systolic_bp")), …`
`  };`
	입력칸 값을 name으로 꺼내 서버가 기대하는 모양(HomeIn)의 객체를 만든다. `num()` 은 빈칸이면 null, 아니면 숫자.

`  const res = await fetch("/home-measurements", {`
`  method: "POST",`
`  headers: { "Content-Type": "application/json" },`
`  body: JSON.stringify(body) });`
	POST 요청. 헤더로 "본문은 JSON"이라고 알리고, `JSON.stringify` 로 객체를 JSON 글자로 바꿔 보낸다.

`  if (res.ok) { $("formDialog").close(); e.target.reset(); loadAll(); }` `  else { showMsg("저장 실패: " + errText(await readJson(res)), false); }`	성공하면 창을 닫고 입력칸을 비우고 다시 불러온다. 실패하면 서버가 보낸 이유를 보여준다.

14.6 Chart.js, localStorage, Notification

Chart.js · 그래프

```
bpChart = new Chart($("bpChart"), {          // <canvas id="bpChart">에 그린다
  type: "line",
  data: { datasets: [
    { label: "수축기", data: [{ x: "2026-10-27", y: 124 }, …] },
    { label: "이완기", data: pt("diastolic_bp") } ]},
  options: { scales: { x: { type: "time" }, y: { min: 50, max: 180 } } },
});
```

데이터를 새로 그리기 전에 `bpChart.destroy()` 로 이전 그래프를 지워야 합니다. 지우지 않으면 같은 canvas에 그래프가 겹쳐 오류가 납니다. x축을 시간(`type: "time"`)으로 쓰려면 날짜 어댑터(chartjs-adapter-date-fns)를 함께 불러옵니다.

localStorage · 브라우저 저장소

```
const store = {
  get(k) { try { return JSON.parse(localStorage.getItem(k) || "[]"); } catch { return []; } },
  set(k, v) { try { localStorage.setItem(k, JSON.stringify(v)); } catch {} },
};
```

localStorage는 브라우저 안에 문자열을 저장하는 공간입니다. 이음은 "이미 본 알림 id 목록"(`ieum-seen-D-001`)과 "이미 띄운 브라우저 알림 id 목록"(`ieum-pushed-D-001`)을 저장합니다. 그 브라우저에만 남고 서버로 가지 않으므로, 다른 기기에서 열면 다시 새 알림으로 보입니다. 사생활 보호 모드 등에서는 막힐 수 있어 `try` 로 감쌌습니다.

Notification · 브라우저 알림

```
const p = await Notification.requestPermission();   // "granted" | "denied" | "default"
new Notification(`이음 · ${n.title}`, { body: n.body, tag: n.id });
```

권한 요청은 반드시 사용자가 버튼을 눌렀을 때 해야 합니다(이음의 "알림 받기" 버튼). 이 방식은 **이음 페이지가 열려 있을 때만** 알림을 띄울 수 있습니다. 앱을 닫아도 알림을 받으려면 서비스 워커와 Web Push(서버가 푸시를 보내는 구조)가 필요합니다.

14.7 스트리밍 응답을 화면에서 읽기

안내 Agent는 답을 한 줄씩(NDJSON, 11.6절) 보냅니다. `res.json()`은 응답이 끝까지 와야 읽을 수 있으므로, 대신 **조각이 오는 대로** 읽습니다.

코드 (donor.html · ask 일부)	해설

`const reader = res.body.getReader();`
`const decoder = new TextDecoder();`
`let buf = "";`
	응답 본문을 조각 단위로 읽는 도구. 조각은 바이트라서 TextDecoder로 글자로 바꾼다.

`while (true) {`
`  const { value, done } = await reader.read();`
`  if (done) break;`
	다음 조각이 올 때까지 기다렸다가 받는다. 끝나면 반복을 멈춘다.

`  buf += decoder.decode(value, { stream: true });`
`  const lines = buf.split("\n");`
`  buf = lines.pop();`
	조각이 줄 중간에서 끊길 수 있으므로, 마지막 줄은 다음 조각과 합치려고 남겨 둔다.

`  for (const line of lines) {`
`  const ev = JSON.parse(line);`
`  if (ev.type === "delta") { text += ev.text; show(); }`
`  if (ev.type === "replace") { text = ev.text; show(); }`
`  }`
`}`
	한 줄 = 이벤트 하나. 새 글자는 이어 붙이고, 출력 검사에 걸린 교체 이벤트면 통째로 바꾼 뒤 말풍선을 다시 그린다.

말풍선 아래에는 `meta` 이벤트의 `mode`에 따라 "AI 답변 · 내 기록 기준", "내 기록에서 바로 확인", "이식센터 안내문", "일반 대화 · 문장 분류 모델", "안전 안내"가 표시되어, 어떤 방식으로 답했는지 공여자도 알 수 있습니다. 화면을 열자마자 `/chat/warmup`을 기다리지 않고 호출해 AI 모델을 미리 준비시킵니다.

꼭 알아야 할 것

- HTML = 구조, CSS = 모양, JS = 동작. id는 JS가 찾을 때, class는 CSS가 꾸밀 때.
- `fetch`로 요청 → `res.ok` 확인 → `res.json()`으로 데이터. 둘 다 `await`.
- 404·500은 catch로 가지 않는다. catch는 서버에 연결조차 안 될 때.
- 브라우저 검사는 편의일 뿐이고, 진짜 검사는 서버와 데이터베이스에서 한다.
- localStorage는 그 브라우저에만 남는다. 여러 기기·사람이 공유할 데이터는 서버 DB에.

15장 Git과 GitHub로 이력 관리·공개하기

Git은 파일의 변경 이력을 저장하는 도구입니다. 저장한 시점(커밋)마다 무엇이 바뀌었는지 남아서, 잘못 고쳤을 때 이전 상태로 돌아갈 수 있습니다. GitHub는 Git 저장소를 인터넷에 올려 두는 서비스입니다.

15.1 세 영역

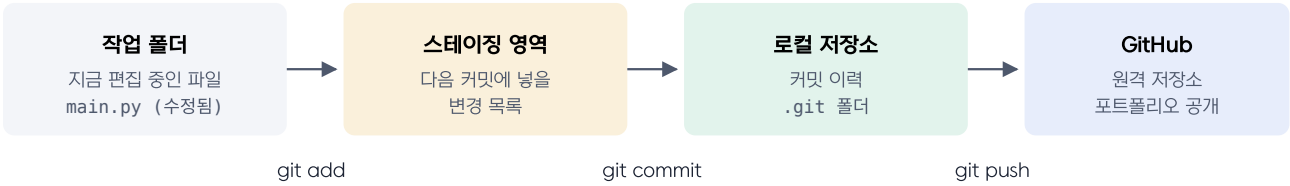


그림 15-1 파일 변경이 기록되기까지 거치는 영역.

15.2 명령

명령	하는 일
`git init`	현재 폴더를 Git 저장소로 만든다(한 번만). `.git` 폴더가 생긴다.
`git status`	바뀐 파일, 스테이징된 파일 확인. 가장 자주 쓴다.
`git add .`	바뀐 파일 전부를 스테이징. `.`은 현재 폴더 전체.
`git commit -m "메시지"`	스테이징된 변경을 하나의 기록으로 저장.
`git log --oneline`	커밋 이력을 한 줄씩 보기. 앞의 7자리(예: `034e060`)가 커밋 id.
`git diff`	아직 스테이징하지 않은 변경 내용 보기.
`git remote add origin 주소` → `git push -u origin main`	GitHub 저장소를 연결하고 올리기.

확인 방법 · 커밋 메시지 쓰기

한 커밋에는 한 가지 변경을 담고, 무엇을 왜 바꿨는지 적습니다. 예: `측정 테이블을 자가 기록과 검사 결과로 분리`, `검사 결과 입력 시 추적검사 자동 완료`. 기능 하나가 동작할 때마다 커밋하면, 다음 작업에서 문제가 생겨도 그 지점으로 돌아갈 수 있습니다.

15.3 .gitignore

```
.venv/
.env
__pycache__/
```

`.gitignore`에 적힌 파일과 폴더는 Git이 무시합니다.

- `.venv/` : 용량이 크고, `pip install`로 다시 만들 수 있다.
- `.env` : 비밀번호. 절대 올리지 않는다.
- `__pycache__/` : Python이 자동으로 만드는 임시 파일.

주의

`.gitignore`는 **아직 커밋되지 않은** 파일에만 효과가 있습니다. .env를 이미 한 번 커밋했다면 .gitignore에 추가해도 이력에 남아 있으므로, `git rm --cached .env`로 추적을 끊고 비밀번호도 바꿔야 합니다. GitHub에 올리기 전 `git status`에 .env가 보이지 않는지 반드시 확인합니다.

다른 사람이 프로젝트를 실행할 수 있게 하려면 설치할 패키지 목록을 파일로 남깁니다: `pip freeze > requirements.txt`. 받는 사람은 `pip install -r requirements.txt`로 같은 환경을 만듭니다. .env 대신 값을 비운 `.env.example`을 함께 올리면 어떤 설정이 필요한지 알 수 있습니다.

15.4 GitHub에 올리기: 원격 저장소

명령	하는 일
`git remote add origin 주소`	GitHub 저장소를 `origin`이라는 이름으로 등록 (한 번만)
`git remote set-url origin 주소`	이미 등록된 `origin`의 주소 바꾸기 ("remote origin already exists" 오류일 때)
`git remote -v`	등록된 원격 주소 확인
`git push -u origin master`	처음 올릴 때. `-u`로 연결해 두면 다음부터는 `git push`만
`git branch -r`	원격 브랜치 목록. `git fetch --prune`으로 지워진 브랜치 정보 정리
`git rm -r --cached .vscode`	Git 추적에서만 빼고 내 PC 파일은 남김
`git rm app.py`	Git과 PC에서 함께 삭제 (기록에서는 되살릴 수 있음)

주의 · 실제로 겪은 것

- GitHub에서 저장소를 만들 때 "Add a README"를 체크하면 `main` 브랜치가 생기고, 내 코드는 `master`에 올라가 **브랜치가 두 개**가 된다. → Settings → General → Default branch를 `master`로 바꾸고 `main` 삭제.
- PowerShell에 명령을 붙여 넣을 때 줄 이어 쓰기 표시 `>>`까지 복사하면 오류가 난다. 한 줄씩 붙여 넣는다.
- 공개(Public) 전에 파일 목록에 `.env`가 없는지 눈으로 확인하고, `git ls-files .env` · `git log --all --.env`가 비어 있는지 확인한다.

15.5 README: 저장소의 첫 화면

README.md는 GitHub 저장소 첫 화면에 표시되는 설명서입니다. 채용 담당자는 코드보다 README를 먼저, 대부분은 README만 봅니다. 이음 README의 순서는 다음과 같습니다.

순서	내용
소개	한 줄 설명, 만든 이유(CRC 경험), 데모·합성 데이터라는 고지
화면	공여자 앱 · 안내 Agent 대화 · 의료진 화면 캡처 (`docs/images/`)
주요 기능 · 구조 · DB	기능 표, Mermaid로 그린 구조도와 테이블 관계도 (GitHub가 그림으로 그려 줌)
안내 Agent 설계	질문 분류 5단계, 안전 설계, **실제로 겪은 문제와 해결**, 측정 수치
ML · DL	평가 방법과 결과, 단순 모델과 비슷했다는 사실, 합성 데이터의 한계
실행 방법 · 구조 · API	다른 사람이 따라 실행할 수 있는 명령, 폴더 설명, API 표
한계와 다음 단계	로그인·권한, 의료진 검토, 실제 데이터, 개인정보, 푸시 알림

````
```mermaid
erDiagram
    donors ||--o{ followups : "추적검사 일정"
    donors ||--o{ home_measurements : "자가 기록"
```
````

꼭 알아야 할 것

- `git add` → `git commit` → `git push` : 작업 폴더 → 스테이징 → 로컬 저장소 → GitHub.
- 기능 하나가 동작하면 바로 커밋. 커밋이 없으면 되돌아갈 지점도 없다.
- .env는 .gitignore로 제외. 커밋 전 `git status` 로 확인.
- 공개 저장소는 비공개로 먼저 올려 확인한 뒤 공개한다. README가 포트폴리오의 첫인상이다.

PART 6

정리

이음을 만드는 동안 실제로 만난 오류, 반드시 설명할 수 있어야 하는 항목, 용어, 그리고 확인 문제입니다. 문제는 답을 보지 않고 먼저 써 본 뒤 맨 뒤의 정답과 맞춰 봅니다.

16장 실제로 만난 오류와 해결

17장 꼭 알아야 할 것 체크리스트

18장 용어 사전

19장 확인 문제 31개와 정답

16장 실제로 만난 오류와 해결

오류 메시지는 대부분 원인을 직접 알려 줍니다. 마지막 줄부터 읽고, 그 메시지가 어느 계층(터미널, 서버, 데이터베이스, 브라우저)에서 나왔는지 먼저 구분합니다.

증상 · 메시지	원인	해결
CREATE TABLE에서 문법 오류	열 자리에 자료형 대신 값(120, D-001)을 적음	열마다 "날짜·정수·소수·글자"를 먼저 정하고 DATE · INTEGER · NUMERIC · TEXT로 적는다.
`Could not import module "main"`	main.py가 저장되지 않았거나 현재 폴더에 없음	`dir` 로 확인. 없으면 `New-Item main.py` 후 내용 저장(Ctrl + S).
파일이 안 보임, .env가 안 읽힘	파일을 `.venv` 폴더 안에 저장	main.py와 같은 프로젝트 최상위 폴더로 옮긴다.
.env를 더블클릭하자 다른 프로그램(AnySign)이 열림	확장자 연결 프로그램 문제	.env는 VS Code에서만 연다.
"서버에 연결할 수 없습니다", connection refused	uvicorn 서버가 꺼져 있음	별도 터미널에서 `uvicorn main:app --reload` 를 켜 둔다.
화면은 나오는데 데이터가 비어 있음, JS 동작 안 함	HTML을 복사·붙여넣기하다 뒷부분이 잘림	파일 전체를 받아 덮어쓴다. 파일 끝 `</html>` 이 있는지 확인.
소변 단백 값이 `string` 으로 저장됨	/docs 예시값을 그대로 실행, 서버 검사 없음	잘못된 행 삭제 후 `Literal[...]` 로 선택지 제한.
미래 날짜, 같은 날 중복 기록	검사 규칙 없음	`field_validator` 로 미래 날짜 거절, `UNIQUE(donor_id, 날짜)` + `ON CONFLICT DO UPDATE` .
500 Internal Server Error (새 API)	migrate_split.sql · seed_demo.sql을 아직 실행하지 않아 테이블이 없음	pgAdmin에서 SQL 파일 실행. 터미널(uvicorn)의 마지막 오류 줄에 `relation "…" does not exist` 가 보인다.
화면이 가로로 밀려 스크롤이 생김	그리드 칸이 내용 크기만큼 늘어남	`minmax(0, 1fr)` , `min-width: 0` .
엑셀 내려받기 500 오류	pandas만 설치하고 openpyxl은 설치하지 않음	`pip show openpyxl` 로 확인 → `pip install openpyxl` . 터미널 오류에는 `No module named 'openpyxl'` .
VS Code에 "Select an environment manager" 창	VS Code가 새 가상환경을 만들자고 제안	`Esc` 로 닫는다. 이미 `.venv` 가 있으므로 Quick Create를 고르지 않는다.
`'ollama' 용어가 … 인식되지 않습니다`	Ollama가 설치되지 않았거나, 설치 전에 연 터미널이라 새 경로를 모름	설치 후 VS Code를 완전히 껐다가 다시 켠다.
Another Ollama installer is running	설치 프로그램을 두 번 실행	취소하고 먼저 실행한 설치가 끝날 때까지 기다린다.
새 모델 가중치(.npz)나 .env를 바꿨는데 반영 안 됨	`--reload` 는 .py 파일이 바뀔 때만 다시 시작	`Ctrl + C` 후 서버를 다시 켠다.
로컬 AI가 혈압을 "정상 범위"라고 지어냄	작은 모델이 도구를 부르지 않음(할루시네이션)	자료 주입 + 출력 검사 (11.4절)
로컬 AI 답이 1분 가까이 걸림	CPU에서 프롬프트 읽기가 대부분	프롬프트 축소 · 미리 읽혀 두기 · 스트리밍 (11.5절)
`HTTP 401 Invalid API Key`	Groq가 아닌 곳의 키 (`gsk_` 로 시작하지 않음)	console.groq.com에서 다시 발급, `/health` 로 확인
`HTTP 404 model_not_found`	설정한 모델이 서비스 종료됨	쓸 수 있는 모델로 자동 변경 (11.8절)
`remote origin already exists`	`git remote add` 를 두 번 실행	`git remote set-url origin 주소`
GitHub에 브랜치가 두 개 (main, master)	저장소를 만들 때 README 추가를 체크	기본 브랜치를 master로 바꾸고 main 삭제 (15.4절)

확인 방법 · 500 오류가 났을 때 보는 곳

브라우저에는 "500"만 보이고 이유가 보이지 않습니다. 이유는 **uvicorn을 켜 둔 터미널**에 출력됩니다. 긴 Traceback의 **맨 아래 줄**이 실제 원인입니다(예: `psycopg.errors.UndefinedTable` , `KeyError` , `TypeError: 'NoneType' object is not subscriptable`).

17장 꼭 알아야 할 것 체크리스트

각 항목을 소리 내어 설명할 수 있으면 체크합니다. 면접에서 "이 프로젝트에서 무엇을 했나"에 대한 답의 재료입니다.

17.1 구조 · 데이터베이스 · 서버

구조

- [ ] 브라우저 · FastAPI · PostgreSQL 세 부분의 역할과, 그 사이에 오가는 형식(HTTP+JSON, SQL)
- [ ] 브라우저가 데이터베이스에 직접 접속하지 않는 이유 세 가지
- [ ] GET과 POST의 차이, 상태 코드 200 · 404 · 422 · 500의 의미

데이터베이스

- [ ] 테이블 · 행 · 열 · 스키마 · 기본키 · 외래키의 정의
- [ ] 이음 테이블 4개와 관계(1:N), 측정값을 두 테이블로 나눈 이유
- [ ] 제약조건 5가지(PRIMARY KEY, NOT NULL, UNIQUE, REFERENCES, DEFAULT)가 막는 것
- [ ] NULL의 의미와 `IS NULL`로 찾아야 하는 이유
- [ ] SELECT 절의 순서, JOIN과 LEFT JOIN의 차이, GROUP BY
- [ ] UPSERT(ON CONFLICT)의 동작과 EXCLUDED의 의미
- [ ] 트랜잭션(전부 반영 또는 전부 취소)과 migrate_split.sql에서 쓴 이유
- [ ] SQL 인젝션과 `%s` 매개변수로 막는 방법

서버

- [ ] 라우트(`@app.get`)와 경로 매개변수(`{donor_code}`)
- [ ] Pydantic 모델의 역할, `Field(ge, le)` · `Literal` · `field_validator`
- [ ] `fetchone()`과 `fetchall()`의 차이, None 처리
- [ ] `with get_conn() as conn`이 하는 일(연결 정리, COMMIT/ROLLBACK)
- [ ] visit_status의 네 가지 상태와 판정 순서, ±60일 기간
- [ ] 검사 결과 입력 시 추적검사가 자동 완료되는 과정
- [ ] 알림을 저장하지 않고 계산하는 이유, id 규칙

17.2 pandas · 규칙 · AI · 화면과 도구

pandas

- [ ] DataFrame · Series · 인덱스 · NaN의 뜻
- [ ] groupby + agg로 공여자별 최근 eGFR 구하기
- [ ] crosstab + reindex로 "공여자 × 12주" 기록 표를 만드는 과정과 reindex가 필요한 이유
- [ ] merge(how="left")와 SQL LEFT JOIN의 관계
- [ ] BytesIO · ExcelWriter · StreamingResponse로 엑셀을 내려받게 하는 흐름

규칙 · AI · 예측

- [ ] 경고 규칙 6가지와 단계, 기준값을 상수로 모은 이유, 공여자에게는 혈압 경고만 보이는 이유
- [ ] 안내 Agent 질문 분류 5단계와 각 단계가 답하는 방식
- [ ] 안전장치 세 겹: 응급 먼저(단어+DL) · 조회 범위 고정 · 출력 검사
- [ ] 작은 모델의 할루시네이션을 자료 주입으로 막은 과정
- [ ] 느린 원인(프롬프트 읽기)과 해결: 축소 · 미리 읽혀 두기 · 스트리밍
- [ ] 로컬(Ollama)과 클라우드(Groq)의 장단점, 개인정보 관점
- [ ] 지도학습 · 특징 · 라벨, 교차검증, AUC · Brier, 과적합
- [ ] 로지스틱 회귀의 표준화 → 점수 → 시그모이드, 계수로 이유 보여 주기
- [ ] 합성 데이터 결과의 한계
- [ ] 신경망 층 · ReLU · softmax · 손실 · 역전파, EmbeddingBag 구조
- [ ] GroupKFold가 필요한 이유, 정밀도 · 재현율 · 기준값

화면과 도구

- [ ] HTML · CSS · JS의 역할, id와 class의 차이
- [ ] fetch + async/await + res.ok + res.json()의 흐름
- [ ] 저장 버튼부터 화면 갱신까지 7단계와 세 번의 검사
- [ ] localStorage와 서버 DB의 차이, Notification API의 한계
- [ ] 가상환경을 쓰는 이유, .env와 .gitignore
- [ ] git add · commit · push의 의미
- [ ] 스트리밍 응답을 reader로 읽는 방법
- [ ] 원격 저장소 등록 · push · 브랜치 정리, 공개 전 .env 확인, README 구성

18장 용어 사전

가나다 · 알파벳 순.

18.1 가나다 순

용어	뜻
가상환경 (venv)	프로젝트 전용 Python 설치 공간
과적합	학습 데이터에만 맞고 새 데이터에서는 틀리는 상태
교차검증	데이터를 여러 조각으로 나눠 번갈아 평가하는 방법
기본키 (PK)	테이블에서 각 행을 구분하는 고유하고 비어 있지 않은 값
데코레이터	함수 위에 `@` 로 붙여 함수에 기능을 더하는 문법. `@app.get`
라벨	지도학습에서 맞혀야 하는 정답
라우트	주소 + 메서드와 실행할 함수의 연결
로지스틱 회귀	점수를 시그모이드로 확률로 바꾸는 분류 모델
마이그레이션	이미 데이터가 있는 데이터베이스의 구조를 바꾸는 작업. migrate_split.sql
반응형	화면 크기에 따라 배치가 바뀌는 웹 디자인
브랜치	작업 흐름을 나눠 기록하는 줄기. 이음은 master 하나
비동기	응답을 기다리는 동안 다른 일을 멈추지 않고 진행하는 실행 방식
스키마	테이블 구조(열, 자료형, 제약조건)의 정의
스트리밍	응답을 다 만들기 전에 생기는 대로 조금씩 보내는 방식
시그모이드	어떤 수든 0~1 사이로 바꾸는 S자 함수
시드 데이터	개발·시연용으로 미리 넣어 두는 데이터. seed_demo.sql
엔드포인트	API에서 요청을 받는 하나의 주소. `/labs`
역전파	손실을 줄이는 방향으로 신경망 가중치의 기울기를 계산하는 방법
외래키 (FK)	다른 테이블의 기본키를 가리키는 열
원격 저장소	GitHub처럼 인터넷에 있는 Git 저장소. origin
재현율	실제 양성 중 모델이 잡아낸 비율
정규화	중복 없이 정보를 한 곳에만 두도록 테이블을 나누는 설계
정밀도	모델이 양성이라고 한 것 중 실제 양성의 비율
제약조건	테이블이 스스로 지키는 데이터 규칙
커밋	Git에 저장한 변경 기록 1건 / DB에서 트랜잭션을 확정하는 것
쿼리	데이터베이스에 보내는 요청 문장
토큰	LLM이 읽고 쓰는 글자 조각 단위
트랜잭션	전부 반영되거나 전부 취소되는 명령 묶음
특징	모델에 넣는 입력값 (feature)
프레임워크	정해진 구조 위에 내 코드를 채워 넣는 개발 틀. FastAPI
프롬프트	LLM에 넣는 지시문과 질문 전체
할루시네이션	모델이 사실이 아닌 내용을 그럴듯하게 지어내는 것
합성 데이터	실제 데이터 대신 규칙과 난수로 만든 가짜 데이터
혼동 행렬	정답과 예측을 교차해 센 표

18.2 알파벳 순

용어	뜻
Agent	LLM이 도구를 골라 쓰며 일을 처리하는 구조
API	프로그램끼리 요청·응답하는 약속된 창구
ASGI 서버	FastAPI 같은 Python 웹 앱을 실행하는 서버 프로그램. uvicorn
AUC	위험이 높은 사람에게 더 높은 점수를 줄 확률. 0.5 찍기 ~ 1.0 완벽
BytesIO	메모리 안의 파일. 디스크에 저장하지 않고 파일 내용을 만들 때 사용
crosstab	두 열의 값 조합별 개수를 센 교차표 (pandas)
CRUD	Create · Read · Update · Delete. 데이터 조작의 네 가지
DataFrame	행과 열로 된 pandas의 표 객체
DDL · DML	구조를 다루는 SQL(CREATE 등) · 데이터를 다루는 SQL(SELECT 등)
DOM	HTML 문서를 JS에서 다룰 수 있게 만든 객체 구조
EmbeddingBag	조각 번호마다 벡터를 찾아 평균 내는 PyTorch 층
ERD	테이블과 그 관계를 그린 그림
Groq	공개 LLM을 빠르게 실행해 주는 클라우드 서비스
groupby	같은 값끼리 나눠 묶음마다 계산 (pandas, SQL의 GROUP BY)
GroupKFold	같은 그룹(문장 틀)이 학습과 평가에 섞이지 않게 나누는 교차검증
HTTP	브라우저와 서버가 주고받는 통신 규칙
JSON	`{"이름": 값}` 형태의 데이터 교환 형식
LLM	대규모 언어 모델
merge	기준 열로 두 표를 합치기 (pandas, SQL의 JOIN)
NaN · NaT	pandas의 "값 없음" 표시 (숫자 · 날짜)
NDJSON	한 줄에 JSON 하나씩 쓰는 형식
NULL	값이 없음. 0이나 빈 문자열과 다름
Ollama	내 컴퓨터에서 LLM을 실행하는 프로그램
openpyxl	엑셀(.xlsx) 파일을 읽고 쓰는 Python 패키지. pandas가 엑셀을 쓸 때 사용
ORM	SQL 대신 클래스로 DB를 다루는 방식(SQLAlchemy 등). 이음은 쓰지 않고 SQL을 직접 작성
pandas	표 데이터를 다루는 Python 라이브러리
psycopg	Python에서 PostgreSQL에 접속하는 라이브러리(버전 3)
Pydantic	자료형과 규칙으로 입력값을 검사하는 Python 라이브러리
PyTorch	딥러닝 모델을 만들고 학습하는 Python 라이브러리
README	저장소 첫 화면에 보이는 설명서
ReLU	음수는 0, 양수는 그대로 두는 활성화 함수
scikit-learn	머신러닝 모델과 평가 도구를 모은 Python 라이브러리
seed	난수의 시작점. 고정하면 결과가 재현됨
softmax	점수들을 합이 1인 확률로 바꾸는 함수
SQL 인젝션	입력값이 SQL 명령으로 해석되게 만드는 공격
UPSERT	있으면 수정, 없으면 입력. `ON CONFLICT DO UPDATE`

19장 확인 문제

답은 빈칸에 직접 써 봅니다. 정답은 다음 쪽에 있습니다.

19.1 문제 01 ~ 16

01 브라우저가 데이터베이스에 직접 접속하지 않고 서버를 거치는 이유를 두 가지 쓰시오.

02 혈압을 300으로 저장하려 할 때 서버가 돌려주는 상태 코드와, 그 검사를 하는 코드의 위치는?

03 다음 열에 알맞은 자료형을 쓰시오. ① 기증일 ② 체중 61.5 ③ 공여자 코드 D-001 ④ 수축기 혈압

04 `completed_date`에 NOT NULL을 붙이지 않은 이유는?

05 아직 받지 않은 추적검사를 찾는 WHERE 조건을 쓰시오. (`= NULL`을 쓰면 안 되는 이유도)

06 `lab_results.donor_id`에 99를 넣었더니 오류가 났다. 어떤 제약조건 때문인가?

07 D-002의 자가 기록을 측정일 최신순으로 3건 조회하는 SQL을 쓰시오. (donor_id = 2)

08 검사 결과 목록에서 JOIN 대신 LEFT JOIN을 쓴 이유는?

09 `ON CONFLICT (donor_id, measured_at) DO UPDATE SET systolic_bp = EXCLUDED.systolic_bp`에서 EXCLUDED는 무엇을 가리키는가? 이 문장이 동작하려면 테이블에 무엇이 있어야 하는가?

10 migrate_split.sql을 BEGIN ~ COMMIT으로 감싼 이유는?

11 다음 코드의 문제점과 고친 코드를 쓰시오. `conn.execute(f"SELECT * FROM donors WHERE donor_code = '{code}'")`

12 `uvicorn main:app --reload`에서 main, app, --reload는 각각 무엇인가?

13 `@app.get("/donors/{donor_code}/labs")`로 `/donors/D-003/labs` 요청이 오면 함수의 donor_code 값은?

14 `fetchone()`의 결과를 None 확인 없이 `row["egfr"]`로 쓰면, 결과가 없을 때 어떻게 되는가?

15 예정일이 2026-06-15이고 완료일이 없을 때, 오늘이 ① 2026-05-01 ② 2026-07-20 ③ 2026-08-20이면 visit_status의 결과는?

16 visit_status에서 `if completed:`를 맨 앞에 두는 이유는?

19.2 문제 17 ~ 31

17 추적검사 상태를 테이블에 저장하지 않고 요청 때마다 계산하는 장점은?

18 fetch에서 서버가 404를 돌려주면 catch 블록이 실행되는가? 그렇지 않다면 어떻게 확인하는가?

19 "이미 본 알림"을 localStorage에 저장했을 때의 한계는?

20 .env를 GitHub에 올리지 않기 위해 해야 할 일과, 이미 커밋했다면 해야 할 일은?

21 DataFrame과 Series의 차이를 쓰고, `df["egfr"]` 와 `df[["donor_code", "egfr"]]` 는 각각 무엇을 돌려주는지 쓰시오.

22 weekly_records에서 crosstab 뒤에 `reindex(index=codes, columns=week_index, fill_value=0)` 를 하지 않으면 어떤 문제가 생기는가?

23 엑셀 내보내기에서 `buffer.seek(0)` 을 빠뜨리면 어떻게 되는가? 그리고 파일을 디스크 대신 BytesIO에 만드는 이유는?

24 경고 규칙의 기준값(135/85 등)을 상수로 한 곳에 모아 둔 이유는? 그리고 공여자 앱에는 왜 혈압 경고만 보여 주는가?

25 안내 Agent가 질문을 처리하는 5단계를 순서대로 쓰고, LLM까지 가는 질문은 어떤 질문인지 쓰시오.

26 작은 로컬 모델이 혈압을 "120/80, 정상 범위"라고 지어낸 문제를 막은 두 가지 방법은?

27 로컬 LLM의 첫 응답이 59초 걸린 주된 원인과, 3.5초로 줄인 방법 두 가지를 쓰시오.

28 로컬 모델(Ollama)과 클라우드 모델(Groq)을 개인정보 관점에서 비교하시오.

29 이탈 위험 모델에서 교차검증 AUC 0.812와 평가용 AUC 0.803이 비슷하다는 것은 무엇을 뜻하는가? 이 숫자를 실제 성능이라고 말할 수 없는 이유는?

30 문장 분류 모델을 GroupKFold로 평가한 이유는?

31 응급 분류에서 정밀도와 재현율 중 무엇이 더 중요한가? 응급 확률 기준값을 0.5보다 낮게 두면 어떻게 되는가?

19장 정답

틀린 문제는 괄호 안의 절로 돌아가 다시 읽습니다.

19.3 정답 01 ~ 16

01 ① DB 비밀번호를 브라우저 코드에 둘 수 없다(누구나 볼 수 있음). ② 서버가 저장 전에 값을 검사한다. ③ 상태 판정 같은 규칙을 한 곳에서 계산해 모든 화면이 같은 결과를 본다. (1.1)

02 422. HomeIn의 `Field(None, ge=50, le=260)` (Pydantic). 함수는 실행되지 않는다. (7.3)

03 ① DATE ② NUMERIC(5,2) ③ TEXT ④ INTEGER (3.2)

04 일정을 만들 때는 아직 검사를 받지 않았으므로 값이 없어야 한다. NULL = 아직 안 받음. (4.2)

05 `WHERE completed_date IS NULL`. NULL은 "알 수 없는 값"이라 `=` 비교 결과가 참이 아닌 NULL이 되어 아무 행도 나오지 않는다. (3.4)

06 외래키(REFERENCES donors). donors에 donor_id 99가 없다. (3.3)

07 `SELECT measured_at, systolic_bp, diastolic_bp, weight_kg FROM home_measurements WHERE donor_id = 2 ORDER BY measured_at DESC LIMIT 3;` (5.1)

08 추적검사와 연결되지 않은 결과(followup_id가 NULL)도 목록에 나와야 하기 때문. JOIN이면 짝 없는 행이 빠진다. (5.3)

09 충돌 때문에 넣지 못한 새 값(방금 INSERT하려던 행). 충돌을 판정할 UNIQUE 제약조건 `UNIQUE(donor_id, measured_at)` 이 있어야 한다. (5.5)

10 테이블 생성 · 데이터 복사 · 예전 테이블 삭제 중 하나라도 실패하면 전부 취소해, 데이터가 절반만 옮겨지거나 예전 테이블만 사라지는 상태를 막기 위해. (5.6)

11 입력값이 SQL 문장에 그대로 들어가 SQL 인젝션이 가능하다. `conn.execute("SELECT * FROM donors WHERE donor_code = %s", (code,))` (5.8)

12 main = main.py 파일, app = 그 안의 FastAPI 객체 변수, --reload = 코드 저장 시 자동 재시작(개발용). (2.4)

13 `"D-003"` (7.2)

14 None에 `["egfr"]`를 쓰게 되어 TypeError가 나고, 요청은 500 오류가 된다. (7.4)

15 기간은 2026-04-16 ~ 2026-08-14. ① window ② window ③ missed (8.1)

16 이미 받은 검사는 날짜와 상관없이 완료여야 한다. 뒤에 두면 받은 검사도 기간이 지났다는 이유로 missed가 된다. (8.1)

19.4 정답 17 ~ 31

17 시간이 지나면 저절로 바뀌는 상태라, 저장하면 매일 갱신하는 작업이 필요하고 갱신이 빠지면 틀린 값이 남는다. 계산하면 항상 현재 날짜 기준으로 정확하다. (8장)

18 실행되지 않는다. 응답은 온 것이므로 `res.ok` (또는 `res.status`)로 확인한다. catch는 서버에 연결하지 못했을 때 실행된다. (14.4)

19 그 브라우저에만 저장된다. 다른 기기나 브라우저에서 열면 같은 알림이 다시 새 알림으로 보이고, 저장소를 지우면 기록이 사라진다. 서버는 이 정보를 알 수 없다. (14.6)

20 .gitignore에 `.env`를 넣고 커밋 전 `git status`로 확인. 이미 커밋했다면 `git rm --cached .env`로 추적을 끊고, 이력에 남은 비밀번호는 바꾼다. (15.3)

21 DataFrame은 행과 열로 된 표, Series는 열 하나. `df["egfr"]`는 Series, `df[["donor_code", "egfr"]]`는 두 열짜리 DataFrame. (9.1, 9.2)

22 기록이 하나도 없는 주는 열이 생기지 않고, 최근 12주에 기록이 없는 공여자는 행이 생기지 않는다. 그러면 기록률의 분모가 12가 아니게 되어 실제보다 높게 나오거나, 그 공여자가 표에서 빠진다. (9.4)

23 읽기 위치가 파일 끝에 있어 빈 파일이 전송된다. BytesIO를 쓰면 서버 폴더에 환자 데이터 파일이 남지 않고, 정리할 파일도 생기지 않는다. (9.6)

24 의료진 검토로 기준이 바뀌면 한 곳만 고치면 화면·엑셀·알림이 모두 따라 바뀐다. 검사 수치 해석을 앱이 직접 알리면 불안을 키울 수 있어 의료진이 전달하도록 했다. (10.1, 10.3)

25 ① 응급(단어 목록 또는 DL) ② 일반 대화(DL) ③ 안내문 주제 ④ 내 기록 질문 ⑤ 그 밖의 자유 질문 → LLM. ①~④에 걸리지 않은 자유 질문만 LLM으로 간다. (11.2)

26 ① 도구 호출 대신 서버가 이 공여자의 자료를 조회해 프롬프트에 넣어 주기(자료 주입) ② 답을 내보내기 전 "정상 범위" 같은 판단 표현을 검사해 교체(출력 검사). (11.3, 11.4)

27 CPU에서 지시문·자료를 읽는 시간(프리필)이 대부분이었다. 프롬프트를 약 550자로 줄이고, 화면을 열 때 공여자 자료를 미리 읽혀 두었다(워밍업). 여기에 스트리밍을 더했다. (11.5)

28 Ollama는 노트북 안에서 돌아 기록이 밖으로 나가지 않는다. Groq는 빠르지만 자료가 외부 서버로 전송되므로 실제 병원 적용에는 정보보호 승인이 필요하다. 그래서 기본값은 로컬, 시연은 클라우드로 설계했다. (11.7)

29 학습에 쓰지 않은 데이터에서도 성능이 유지된다 = 과적합이 없다. 그러나 합성 데이터라서 모델은 데이터를 만들 때 넣은 가정을 다시 찾아냈을 뿐이므로 실제 공여자 성능의 근거가 아니다. (12.2, 12.3)

30 같은 문장 틀에서 나온 문장이 학습과 평가에 함께 들어가면 거의 같은 문장을 맞히게 되어 점수가 부풀려진다. 처음 보는 표현에서의 성능을 보려면 틀 단위로 나눠야 한다. (13.4)

31 재현율. 응급을 놓치는 비용이 오경보보다 크다. 기준을 낮추면 재현율이 오르고(0.707 → 0.767) 오경보도 늘어난다(0.054 → 0.074). (13.5)